



UNIVERSITÀ  
DEGLI STUDI  
FIRENZE

Scuola di Scienze Matematiche, Fisiche e Naturali  
Corso di Laurea in Informatica

Tesi di Laurea

STEGANOGRAFIA GENERATIVA SU  
IMMAGINI:  
ADATTAMENTO DEL PROTOCOLLO METEOR  
TRAMITE L'ARCHITETTURA VQ-GAN

GENERATIVE IMAGE STEGANOGRAPHY:  
ADAPTING THE METEOR PROTOCOL TO THE  
VQ-GAN ARCHITECTURE

MATTEO DICIOTTI

Relatore: Prof. *Daniele Baracchi*

Correlatrice: PhD *Giulia Bertazzini*

Correlatore: Prof. *Daniele Castellana*

Anno Accademico 2025-2026

Matteo Diciotti: *Steganografia generativa su immagini:*  
*adattamento del protocollo Meteor tramite l'architettura VQ-GAN*, Corso di  
Laurea in Informatica, © Anno Accademico 2025-2026

---

## INDICE

---

|                                                        |    |
|--------------------------------------------------------|----|
| Elenco delle figure                                    | 3  |
| 1 Introduzione                                         | 7  |
| 2 Stato dell'arte                                      | 11 |
| 2.1 Meteor                                             | 11 |
| 2.1.1 Contesto e motivazioni                           | 11 |
| 2.1.2 Meccanismo Operativo del Protocollo Meteor       | 15 |
| 2.2 VQ-GAN                                             | 20 |
| 2.2.1 Contesto e motivazioni                           | 20 |
| 2.2.2 Architettura del modello VQ-GAN                  | 20 |
| 2.3 Lavori correlati                                   | 25 |
| 3 Definizioni e terminologia                           | 29 |
| 3.1 Concetti fondazionali della comunicazione          | 29 |
| 3.2 Discipline della comunicazione sicura              | 30 |
| 3.3 Fondamenti di modellazione linguistica             | 31 |
| 3.4 Componenti dei modelli linguistici                 | 32 |
| 3.5 Meccanismi di generazione e controllo              | 33 |
| 3.6 Architetture di reti neurali                       | 34 |
| 3.7 Operazioni fondamentali delle reti neurali         | 34 |
| 3.8 Funzioni obiettivo e metriche di addestramento     | 36 |
| 3.9 Classificatori e reti per la percezione visiva     | 37 |
| 3.10 Architetture per la generazione visuale           | 38 |
| 3.11 Misure e garanzie nei sistemi steganografici      | 39 |
| 4 Architettura del sistema proposto                    | 43 |
| 4.1 Visione d'insieme                                  | 43 |
| 4.1.1 Scelta del modello di generazione delle immagini | 43 |
| 4.2 Idea fondante                                      | 43 |
| 4.3 Flusso operativo del sistema                       | 44 |
| 4.4 Rappresentazione delle immagini e griglia di patch | 45 |
| 4.5 Algoritmo di encoding steganografico               | 46 |
| 4.5.1 Selezione steganografica del token               | 46 |
| 4.5.2 Codifica vincolata di un singolo token           | 46 |
| 4.5.3 Flusso principale di encoding steganografico     | 48 |
| 4.6 Algoritmo di decoding steganografico               | 48 |
| 4.6.1 Estrazione dei bit da un token                   | 48 |
| 4.6.2 Flusso principale di decoding steganografico     | 49 |

|       |                                                                   |    |
|-------|-------------------------------------------------------------------|----|
| 4.7   | Gestione del contesto e sliding window . . . . .                  | 49 |
| 4.8   | Limitazioni teoriche . . . . .                                    | 50 |
| 4.8.1 | Non-determinismo della VQ-GAN . . . . .                           | 50 |
| 4.8.2 | Capacità steganografica . . . . .                                 | 51 |
| 5     | Implementazione del sistema . . . . .                             | 53 |
| 5.1   | Architettura software . . . . .                                   | 53 |
| 5.1.1 | Scelta del linguaggio di programmazione e del framework . . . . . | 53 |
| 5.1.2 | Moduli principali . . . . .                                       | 53 |
| 5.2   | Le classi Steganography Encoder e Steganography Decoder . . . . . | 54 |
| 5.2.1 | SteganographyEncoder . . . . .                                    | 54 |
| 5.2.2 | SteganographyDecoder . . . . .                                    | 55 |
| 5.3   | Costruzione del contesto . . . . .                                | 55 |
| 5.4   | Generazione e salvataggio delle immagini . . . . .                | 55 |
| 5.4.1 | Verifica di codifica . . . . .                                    | 55 |
| 5.4.2 | Salvataggio in formato PNG . . . . .                              | 56 |
| 5.5   | Limitazioni pratiche . . . . .                                    | 56 |
| 5.5.1 | Costi computazionali . . . . .                                    | 56 |
| 5.5.2 | Gestione dei casi di bordo . . . . .                              | 57 |
| 5.5.3 | Riproducibilità dei risultati . . . . .                           | 57 |
| 6     | Risultati sperimentali . . . . .                                  | 59 |
| 7     | Conclusioni e sviluppi futuri . . . . .                           | 61 |
|       | Bibliografia . . . . .                                            | 63 |

---

ELENCO DELLE FIGURE

---

|          |                                                       |    |
|----------|-------------------------------------------------------|----|
| Figura 1 | Alice e Bob attraversano il firewall di Eve . . . . . | 13 |
| Figura 2 | Network Security - the sad truth . . . . .            | 60 |



*"Questo è bello!"*  
*"Eh, nella sua semplicità.."*  
*"Questo è iper-realista."*  
*"E se me lo concedi monocromatico."*  
*"L'artista è Meteor."*  
*"Meteor, già sentito!"*  
*— Aldo, Giovanni e Giacomo*





---

## INTRODUZIONE

---

L'esigenza intrinseca e crescente di garantire comunicazioni che siano contemporaneamente riservate, autentiche e integre rappresenta un pilastro fondamentale su cui si erge l'architettura stessa del mondo digitale contemporaneo. La sicurezza delle informazioni, in passato, è stata prevalentemente affidata a robuste tecniche crittografiche e a protocolli standard ben consolidati, quali il *Transport Layer Security* (TLS) e il suo predecessore *Secure Sockets Layer* (SSL). Questi sistemi sono stati meticolosamente progettati per rendere il contenuto effettivo dei messaggi inaccessibile a qualsiasi entità esterna non in possesso delle chiavi di decifratura appropriate; nel contesto di TLS, ciò si traduce tipicamente nell'esclusione di chiunque non partecipi al processo di negoziazione della connessione (l'handshake).

Tuttavia, l'escalation senza precedenti delle misure di sorveglianza di massa su scala globale e la proliferazione di meccanismi di censura sempre più capillari e sofisticati hanno messo a nudo una vulnerabilità critica, intrinseca e finora difficile da eradicare alla crittografia moderna: la sua stessa visibilità. Sebbene la crittografia riesca efficacemente a mantenere segreto il contenuto dei dati trasmessi, l'elevata entropia e le caratteristiche distintive del traffico cifrato lo rendono facilmente identificabile e distinguibile dalle comunicazioni in chiaro. Questo stato di cose consente a regimi autoritari, a entità statali o a sofisticati sistemi di filtraggio di rete (come i firewall avanzati) di individuare, bloccare o degradare selettivamente le comunicazioni protette, compromettendo così l'effettivo scambio di informazioni sensibili in contesti operativi ostili o sorvegliati.

Di fronte a tali sfide, specialmente in scenari caratterizzati da censura estrema o da ambienti di comunicazione altamente avversari, emerge in modo prepotente la necessità di esplorare e impiegare metodologie steganografiche. La steganografia, come disciplina, si dedica all'arte e alla scienza dell'occultamento di informazioni segrete all'interno di oggetti di

copertura (*cover objects*) che, teoricamente, appaiono del tutto innocui e non sospetti ad un'osservatore esterno. Questi oggetti di copertura possono assumere svariate forme, tra cui testi, immagini, file audio o video. A differenza della crittografia, il cui successo e la cui sicurezza dipendono in larga misura dalla complessità matematica o computazionale necessaria per decifrare il messaggio cifrato in assenza delle chiavi, la sicurezza in ambito steganografico poggia su un principio radicalmente diverso: l'indistinguibilità statistica. L'obiettivo primario è fare in modo che un osservatore esterno non sia minimamente in grado di discriminare tra un oggetto che contiene un messaggio segreto (denominato *stego-object*) e un oggetto analogo che ne è privo.

Le metodologie steganografiche classiche, quali ad esempio la manipolazione dei bit meno significativi (*Least Significant Bit - LSB*) nelle immagini digitali, si sono progressivamente rivelate vulnerabili agli attacchi di crittoanalisi statistica sempre più avanzati e raffinati. Per superare queste limitazioni intrinseche e garantire un livello di sicurezza che sia non solo robusto ma anche dimostrabile, la ricerca più recente ha abbracciato un vero e proprio cambio di paradigma. Questo nuovo orientamento si concentra sull'utilizzo strategico di modelli generativi, in particolare quelli basati su reti neurali profonde.

In questo fertile contesto di ricerca si inserisce **Meteor**, un innovativo algoritmo di steganografia a chiave simmetrica. Meteor sfrutta le potenti capacità computazionali e generative dei modelli di linguaggio di grandi dimensioni (*Large Language Models - LLM*), utilizzando originariamente l'architettura GPT-2, per creare canali di comunicazione occulti e resilienti. A differenza delle tecniche steganografiche tradizionali che modificano un contenuto esistente, Meteor interviene e guida il processo di generazione stessa del contenuto. Esso utilizza la distribuzione di probabilità intrinseca del modello generativo per "pilotare" il campionamento dei token successivi in base al messaggio segreto opportunamente cifrato. Questo approccio garantisce che lo *stego-object* risultante sia statisticamente coerente e indistinguibile dall'output naturale del modello, rendendo la presenza del messaggio nascosto estremamente difficile, se non impossibile, da rilevare attraverso analisi statistiche convenzionali.

Obiettivo della presente tesi triennale è estendere il principio fondamentale di sicurezza steganografica ideato da Meteor, originariamente applicato al dominio testuale, all'ambito delle immagini digitali ad alta risoluzione, definendo un modello **SwE** (*Steganography without Embedment*). Per conseguire questo scopo, viene sfruttata l'architettura **VQ-GAN** (*Vector Quantized Generative Adversarial Network*). La scelta di tale

architettura deriva dalla sua capacità di modellare immagini estremamente complesse attraverso la creazione e l'utilizzo di un vocabolario discreto di "token visivi", memorizzati in un *codebook*. Questa rappresentazione discreta consente l'applicazione di un modello Transformer autoregressivo, ampiamente utilizzato nella generazione sequenziale, per guidare il processo di sintesi dell'immagine. Il lavoro svolto mira pertanto a dimostrare con rigore e sperimentazione come sia possibile incorporare messaggi cifrati direttamente durante la fase di generazione dell'immagine, senza necessità di addestrare un nuovo modello e quindi modificare la distribuzione di probabilità di un modello preesistente. L'obiettivo finale è ottenere immagini non solo visivamente simili a quelle generate dallo stesso modello, ma anche intrinsecamente robuste contro tentativi sofisticati di rilevamento steganografico.

Le implicazioni pratiche e teoriche di questo approccio sono significative per lo sviluppo di strumenti di comunicazione *copyright-resistant*.

Il presente elaborato è stato organizzato per condurre il lettore attraverso un percorso logico e approfondito, percorrendo il seguente schema:

- Il **Capitolo 2** fornisce un'analisi completa e dettagliata del background teorico indispensabile, presentando le architetture di Meteor e VQ-GAN e presentando lo stato dell'arte delle tecniche steganografiche esistenti e i principi fondamentali di funzionamento dei modelli generativi più avanzati, e mostrando con ciò una panoramica di lavori correlati e di ricerche affini che hanno ispirato e guidato lo sviluppo del lavoro di tesi.
- Il **Capitolo 3** presenta le definizioni formali necessarie alla completa comprensione del lavoro di tesi.
- Il **Capitolo 4** presenta l'idea alla base del lavoro di tesi, l'architettura completa del sistema proposto e i limiti teorici e pratici del lavoro svolto. Vengono illustrate in dettaglio le modifiche apportate al meccanismo di campionamento dei token visivi per consentire l'embedding del messaggio segreto cifrato direttamente nel flusso generativo.
- Il **Capitolo 5** descrive l'implementazione pratica del sistema steganografico presentato nel capitolo precedente. Vengono illustrati i dettagli realizzativi: l'architettura software, l'organizzazione del codice e le procedure per la generazione e il salvataggio delle

immagini.

- Il **Capitolo 6** espone in modo esaustivo i risultati delle sperimentazioni condotte. Vengono valutate sia la qualità visiva delle immagini generate, attraverso l'utilizzo di metriche consolidate come la Fréchet Inception Distance (FID), sia la capacità steganografica del sistema, analizzando il payload trasferibile e la robustezza contro tentativi di rilevamento steganografico.
- Infine, il **Capitolo 7** trae le conclusioni finali del lavoro svolto, riassumendo i contributi principali della ricerca e delineando una serie di possibili sviluppi futuri, mirati al miglioramento incrementale dell'efficienza, della robustezza e della sicurezza complessiva del sistema proposto.

---

## STATO DELL'ARTE

---

### 2.1 METEOR: UN FRAMEWORK DI STEGANOGRAFIA GENERATIVA SU TESTO

#### 2.1.1 *Contesto e motivazioni*

La necessità di una solida comprensione teorica, indispensabile per contestualizzare appieno il presente lavoro di ricerca, si articola attraverso una costellazione di concetti interdipendenti che convergono in un'unica opera fondamentale: [28], nominalmente nota come *Meteor*. Questa indagine, che va oltre una mera disamina della steganografia classica e di quella generativa, introduce primariamente un approccio metodologico e, in secondo luogo, un framework robusto ed efficiente deputato alla *generazione di messaggi testuali contenenti informazioni celate*.

La peculiarità di queste informazioni risiede nella loro intrinseca invisibilità non solo all'occhio umano, ma anche, e soprattutto, alle più sofisticate tecniche di steganalisi basate su divergenza statistica. La ragione di tale elusività risiede nella formalizzazione del concetto di *perfectly secure steganography* nel dominio del linguaggio naturale: il testo generato da *Meteor* non manifesta alcuna deviazione statistica significativa rispetto a un testo prodotto autonomamente dal medesimo modello linguistico di base (la cosiddetta *cover distribution*). Come verrà dettagliato in seguito, *Meteor* si avvale di un modello linguistico che ha raggiunto una notevole notorietà, estendendosi ben oltre gli ambiti accademici tradizionali, configurandosi come uno dei primi Transformer a larga scala a influenzare il dibattito pubblico sulla capacità generativa delle IA. Nello specifico, *Meteor* impiega il modello **GPT-2** (Generative Pre-trained Transformer 2), caratterizzato da un'architettura *decoder-only* addestrata su un corpus massivo di dati testuali (WebText). Tuttavia, l'architettura del framework è stata concepita con una notevole flessibilità, risultando agnostica rispetto al modello linguistico sottostante (*Backbone LLM*). Questa modularità

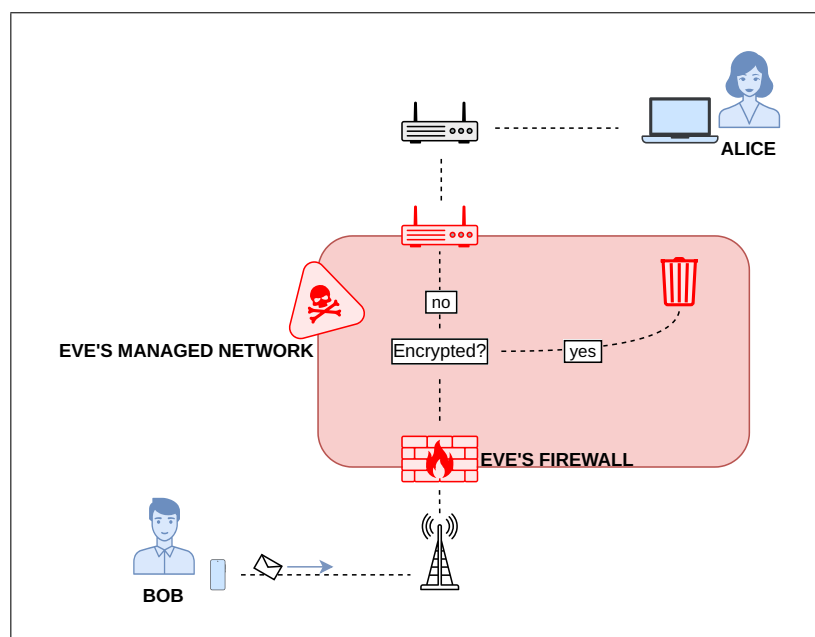
permette a *Meteor* di beneficiare direttamente dei progressi nello stato dell'arte dei modelli linguistici, poiché l'efficacia della steganografia è intrinsecamente legata alla capacità del modello di approssimare la vera distribuzione del linguaggio umano ( $P_{\text{lang}}$ ).

Per una comprensione ancora più profonda, è opportuno esaminare il contesto socio-tecnologico in cui *Meteor* è maturato e le problematiche specifiche che si proponeva di risolvere, con particolare attenzione alla crescente sofisticazione dei sistemi di sorveglianza e censura digitale. In un'epoca in cui la libertà di espressione e la privacy online sono costantemente minacciate da regimi autoritari e da sistemi di filtraggio sempre più avanzati, la necessità di strumenti di comunicazione *copyright-resistant* è diventata più urgente che mai. *Meteor* si inserisce in questo contesto come una risposta innovativa, offrendo un metodo per nascondere informazioni all'interno di testi generati in modo naturale, rendendo estremamente difficile per qualsiasi osservatore esterno rilevare la presenza di un messaggio segreto.

L'essenza di *Meteor* risiede nell'idea innovativa di utilizzare un modello generativo per la creazione di contenuti steganografici, sfruttando la sua capacità di modellare la probabilità condizionata  $P(x_t|x_{<t})$ . In termini pratici, il modello agisce come un sofisticato stimatore di densità: data una sequenza di contesto iniziale (*prompt*), esso fornisce una distribuzione di probabilità dettagliata (solitamente tramite una funzione *softmax* applicata ai *logits*) per tutti i possibili *token* contenuti nel vocabolario  $\mathcal{V}$ . La scelta del *token* da inserire nella sequenza viene effettuata tramite un meccanismo di campionamento che mappa i bit del messaggio segreto in indici del vocabolario, seguendo l'ordinamento indotto dalla distribuzione di probabilità. Questa metodologia trasforma il processo di inferenza in un'operazione di codifica di canale: il testo risultante è statisticamente probabile secondo il modello, ma contiene un messaggio segreto decodificabile univocamente dal destinatario. La decodifica è resa possibile a condizione che il ricevente disponga dello stesso modello linguistico, della medesima sequenza di contesto e di una chiave privata condivisa (*shared secret*), necessaria per inizializzare il generatore di numeri pseudo-casuali o per definire le permutazioni degli indici del vocabolario.

Costruiamo ora uno scenario illustrativo volto a consolidare la comprensione del meccanismo operativo di *Meteor*. Immaginiamo una comunicazione confidenziale tra due entità, Alice (A) e Bob (B), che desiderano scambiarsi informazioni sensibili attraverso un canale pubblico monitorato. L'obiettivo primario è impedire a un osservatore avversario, Eve

(E), di rilevare la presenza stessa della comunicazione (*steganographic undetectability*). Un contesto critico si verifica quando Alice o Bob operano all'interno di infrastrutture soggette a *Deep Packet Inspection* (DPI) o filtraggio governativo stringente. In tali scenari, i messaggi cifrati tramite crittografia standard (come AES o RSA) presentano un'elevata entropia e caratteristiche distintive che li rendono identificabili come traffico cifrato, risultando facilmente etichettabili come traffico non lecito e quindi soggetto a blocco immediato (Figura 1). Un esempio emblematico è il sistema descritto in [50], il "Great Firewall", che impiega euristiche avanzate per discriminare tra traffico legittimo e protocolli di tunneling o cifratura. All'interno di questo quadro ostile, la steganografia generativa supera il limite della crittografia tradizionale: non si limita a proteggere il contenuto, ma nasconde l'esistenza stessa dell'atto comunicativo. Eve, osservando il transito di un testo coerente e stilisticamente appropriato, non ha basi statistiche per etichettare la comunicazione come sospetta.



**Figura 1:** Rappresentazione schematica di una comunicazione tra Alice (A) e Bob (B) in presenza di un osservatore esterno, Eve (E), e di un sistema di filtraggio dei contenuti (Firewall) che analizza i pacchetti.

Sotto il profilo teorico, la progettazione di *Meteor* ha richiesto la risoluzione di due criticità fondamentali intrinseche alla natura del linguaggio naturale.

La prima sfida concerne la definizione di un *sampler steganografico* efficace. L'algoritmo deve mappare il contesto fornito al modello in una

distribuzione di probabilità del token successivo che ricalchi quanto più fedelmente possibile quella del linguaggio naturale. Risulta tuttavia evidente che il linguaggio naturale non sottostia a una distribuzione di probabilità assoluta e invariante: si considerino, a titolo esemplificativo, le divergenze linguistiche intercorrenti tra un testo dei primi del Novecento e uno contemporaneo, o tra la prosa scientifica e quella letteraria. *Meteor* affronta tale complessità adottando un modello di generazione testuale come approssimatore dinamico del linguaggio naturale. Tale approccio consente di simulare efficacemente la distribuzione linguistica in molteplici domini, superando il problema dell'assenza di una distribuzione statistica universale che ha storicamente limitato la steganografia classica. È tuttavia opportuno precisare che l'efficacia di *Meteor* non è intrinsecamente dipendente dalla capacità del modello linguistico sottostante (il *Backbone LLM*) di approssimare la distribuzione reale. Qualora il modello non riuscisse a catturare le sfumature e le complessità semantiche del linguaggio umano, l'oggetto steganografico prodotto rimarrebbe comunque invulnerabile a tecniche di rilevamento avanzate, in quanto ogni output generato da *Meteor* risulta statisticamente indistinguibile da un testo prodotto autonomamente dal medesimo modello in assenza di imposizione nel campionamento. Tale proprietà garantisce una sicurezza steganografica assoluta, fondata sull'indistinguibilità formale tra le distribuzioni degli oggetti generati con e senza codifica del messaggio. Ciò nonostante, l'efficienza steganografica è altresì vincolata alla credibilità percettiva dell'oggetto prodotto: qualora un testo generato presentasse divergenze sistematiche rispetto alle attese statistico-distribuzionali del linguaggio naturale, esso risulterebbe immediatamente sospetto, indipendentemente dalla sua indistinguibilità formale rispetto alla distribuzione del modello. In tal senso, l'efficacia complessiva di *Meteor* rimane correlata alla capacità del modello linguistico sottostante di approssimare fedelmente la distribuzione di probabilità del linguaggio naturale, costituendo tale approssimazione un fattore determinante per la verosimiglianza del contenuto generato.

La seconda sfida è legata alla variabilità dell'entropia testuale. In segmenti caratterizzati da elevata densità semantica o vincoli sintattici stringenti (bassa entropia), la libertà di selezione del token successivo è pressoché nulla. Ad esempio, data la sequenza "*The largest carnivore of the Cretaceous period was the Tyrannosaurus*", il "token" "*Rex*" presenta una probabilità estremamente elevata. Forzare la codifica di bit in tale posizione selezionando un token alternativo produrrebbe un'anomalia semantica macroscopica, facilmente individuabile da un classificatore neurale. Per



ovviare a ciò, *Meteor* introduce una soglia di entropia minima: il sistema analizza la distribuzione di probabilità a ogni passo temporale e determina dinamicamente se la varianza sia sufficiente per ospitare informazione. Se l'entropia è inferiore a un valore critico  $\tau$ , il sistema genera un token senza inserire informazione steganografica. Qualora, invece, l'entropia superi tale soglia, il sistema seleziona i token con probabilità rilevante, escludendo la coda della distribuzione. Attraverso un meccanismo di partizionamento degli indici basato sulle probabilità relative, viene campionato il token i cui estremi dell'intervallo di probabilità presentino una corrispondenza tra i bit più significativi e i bit del messaggio da codificare. Nel caso in cui l'intervallo non permetta la codifica di alcun bit (ovvero quando gli estremi non condividono il prefisso binario), *Meteor* adotta una strategia a bitrate variabile, rinunciando all'inserimento di informazione per quel determinato passo. In fase di decodifica, la conoscenza della distribuzione di probabilità e dei relativi indici consente di determinare univocamente se l'informazione sia stata inserita o meno, potendo recuperare l'intervallo relativo al token selezionato e recuperando i bit più significativi comuni per tutti i valori nell'intervallo. Sebbene tale eventualità sia rara data l'elevata entropia media del linguaggio, questa strategia di codifica adattiva (*adaptive rate*) garantisce che il throughput informativo sia massimizzato esclusivamente dove il linguaggio offre una ridondanza naturale, preservando rigorosamente l'integrità statistica del testo prodotto.

### 2.1.2 Meccanismo Operativo del Protocollo Meteor

L'elemento distintivo fondamentale di Meteor rispetto agli approcci steganografici tradizionali risiede in un radicale spostamento paradigmatico: l'informazione non viene occultata all'interno di un oggetto preesistente mediante alterazioni (*embedding*), bensì impiegata per **orientare il processo di campionamento** in fase di generazione di un nuovo media. Da questo punto di vista, Meteor si discosta nettamente anche da quei sistemi che realizzano generatori steganografici attraverso l'addestramento *ad hoc* di modelli, i quali introducono un'inevitabile componente statistica nel processo di decodifica e alterano le distribuzioni di probabilità dei modelli sottostanti.

Il funzionamento pratico del protocollo si articola nelle seguenti fasi sequenziali:

1. **Cifratura del Messaggio:** Il messaggio segreto viene cifrato utiliz-

zando una chiave simmetrica condivisa tra mittente e destinatario. Questo passaggio è essenziale per trasformare il messaggio in una stringa di bit computazionalmente indistinguibile dal rumore casuale, garantendo che le scelte di campionamento effettuate dal protocollo non presentino pattern statistici rilevabili.

2. **Accesso alla Distribuzione Condizionata:** Meteor si appoggia a un modello generativo pre-addestrato (originariamente GPT-2 il quale possiede un'architettura basata su Transformers, necessità imprescindibile anche per lo sviluppo di modelli nel dominio visivo, quindi anche nel caso della VQ-GAN utilizzata nella nostra implementazione, che vedremo in seguito). Ad ogni passo temporale  $t$ , il modello riceve il contesto dei token già generati e produce una **distribuzione di probabilità**  $P_t$  su tutto il vocabolario dei possibili token successivi.
3. **Campionamento Vincolato:** Invece di campionare casualmente o scegliere il token più probabile, Meteor utilizza i bit del messaggio cifrato come "indice" per selezionare il token dalla distribuzione  $P_t$ . Lo spazio di probabilità viene partizionato e mappato sulle sequenze di bit in modo che il token scelto sia sempre **statisticamente coerente** con il modello, selezionando il token tra quelli più probabili, eliminando la coda della distribuzione sotto una soglia pre-specificata di probabilità. Poiché la scelta avviene all'interno della distribuzione naturale del modello, l'output steganografico rimane indistinguibile da un output generato classicamente dallo stesso modello.
4. **Generazione Iterativa:** Questo processo avviene in modo autoregressivo. Ogni token generato entra a far parte del nuovo contesto per il passo successivo, continuando fino a quando l'intero oggetto non è completo o la sequenza di bit da codificare non è terminata. In quest'ultimo caso, si producono dei token tramite tecniche di campionamento standard fino al completamento sintattico e semantico del media, che questo sia un periodo testuale o un contenuto multimediale. Il troncamento non è netto dopo la codifica dell'ultimo token, ma può essere definito da un carattere speciale che indica la fine del messaggio, o da un limite di lunghezza predefinito. In questo modo, il testo o il multimediale risultante è coerente.
5. **Estrazione e Decodifica:** Il ricevente, possedendo lo stesso modello

generativo e la chiave simmetrica, è in grado di ricreare le medesime distribuzioni di probabilità  $P_t$  per ogni passo del processo. Identificando quali token sono stati effettivamente inclusi nello stego-media ricevuto, il destinatario può risalire ai bit che hanno determinato quelle scelte, ricostruendo il messaggio cifrato e, infine, il segreto originale.

L'efficacia di Meteor risiede nella sua capacità di eliminare le anomalie statistiche che gli strumenti di **steganalisi moderna** solitamente identificano. In questo contesto, il messaggio segreto non è contenuto "nei" dati, ma rappresenta la **ragione deterministica** per cui determinati dati coerenti sono stati generati al posto di altri.

Per comprendere appieno il flusso del protocollo, è utile illustrare uno scenario concreto<sup>1</sup>. Supponiamo che Alice voglia comunicare con il compagno Bob senza che sua madre Eve, esperta di reti di telecomunicazioni e ostile alla loro relazione, lo venga a sapere. Alice ha dimenticato il proprio computer a casa di Bob e deve chiedergli di riportarglielo, ma Eve intercetta ogni comunicazione all'interno della rete domestica. In questo scenario, Alice ha esaurito il traffico dati cellulare e non ha accesso a reti Wi-Fi alternative. Il firewall di casa è stato configurato da Eve per bloccare qualsiasi traffico crittografato end-to-end; i messaggi devono transitare in chiaro per poter superare il gateway. La navigazione esterna è mediata da un proxy che instrada le informazioni tramite un protocollo, ideato dalla stessa Eve, che lascia i contenuti leggibili. Di conseguenza, ogni messaggio inviato o ricevuto da Alice è potenzialmente visibile a sua madre.

In questo contesto, qualsiasi comunicazione cifrata in forma tradizionale verrebbe immediatamente bloccata. Alice e Bob decidono quindi di utilizzare *Meteor*. Alice redige un messaggio segreto: «Mi porteresti il portatile domani per favore, l'ho lasciato sul tavolo di cucina da te.». Utilizzando una chiave simmetrica condivisa (ad esempio la data del loro anniversario), Alice cifra il testo ottenendo una stringa di bit. Il messaggio originale, lungo 87 caratteri in codifica ASCII, corrisponde a 87 byte, ovvero 696 bit. Dopo la cifratura, si ottiene una sequenza di 696 bit che è computazionalmente indistinguibile da una stringa interamente casuale. Poiché l'invio diretto di tale stringa verrebbe intercettato e bloccato da Eve, Alice si affida a *Meteor* per trasformarla.

---

<sup>1</sup> L'esempio proposto rappresenta una forzatura narrativa volta a evitare i classici scenari di sorveglianza bellica o regimi autoritari. Sebbene la trama possa apparire singolare, ogni passaggio tecnico descritto riflette operazioni realmente attuabili a livello pratico.

Eve è abituata a vedere Alice e Bob scambiarsi testi generati da modelli linguistici mentre studiano insieme per l'università. Alice, studentessa di ingegneria informatica, decide di sfruttare questa abitudine: sa che Eve non sospetterà nulla se vedrà transitare nella chat una serie di messaggi relativi alla preparazione dell'esame di Analisi II, scambiandoli per normali interazioni con un LLM. Alice inserisce il messaggio cifrato in *Meteor* e fornisce come contesto iniziale (*prompt*) il testo di un esercizio sullo studio del dominio di una funzione. Il framework genererà un testo che non solo nasconde il messaggio cifrato in modo trasparente, ma risulterà anche semanticamente coerente con il contesto matematico fornito.

L'efficienza di questo processo dipende dalla «precisione» di *Meteor*, ovvero dal numero di bit che possono essere codificati in un singolo token. Questo valore è influenzato dalla dimensione del vocabolario e dalla distribuzione di probabilità dei token. Mentre per modelli come GPT-2 è teoricamente possibile codificare fino a 32 bit per token, in media se ne ottengono circa 15. Tuttavia, in contesti a bassa entropia come un esercizio di analisi, la capacità di codifica può scendere sensibilmente. Supponendo uno scenario cautelativo con una capacità di 5 bit per token (con token mediamente composti da 4 caratteri), per codificare i 696 bit del messaggio Alice dovrà generare un testo di circa 140 token (circa 560 caratteri). Tale lunghezza è perfettamente compatibile con una normale risposta di un assistente virtuale.

Per segnalare la fine della comunicazione, può essere utilizzato un carattere speciale che indichi al destinatario il termine del messaggio, per quanto non sia fondamentale in quanto, in caso di assenza di questo carattere, la decodifica procederà normalmente con l'unica differenza che i token decodificati nella fase finale del messaggio sembreranno casuali (in quanto di fatto lo sono).

L'output che Eve osserverà sarà simile al seguente testo:

---

"Il dominio di una funzione di due variabili e' l'insieme delle coppie  $(x, y) \in \mathbb{R}^2$  per le quali la funzione e' definita. Consideriamo ad esempio  $f(x, y) = \sqrt{1 - x^2 - y^2}$ ; affinche' la radice quadrata sia definita, deve valere  $1 - x^2 - y^2 \geq 0$ , ovvero  $x^2 + y^2 \leq 1$ , che rappresenta il cerchio unitario chiuso nel piano cartesiano. Prendiamo invece  $g(x, y) = \frac{1}{x^2 + y^2 - 1}$ : il denominatore si annulla quando  $x^2 + y^2 = 1$ , quindi il dominio e' tutto  $\mathbb{R}^2$  esclusa la circonferenza di raggio 1. Per  $h(x, y) = \log(x - y)$ , la condizione e'  $x - y$

$> 0$ , ossia il semipiano al di sopra della bisettrice  $y = x$ . In generale, per determinare il dominio di una funzione multivariata, dobbiamo identificare tutte le condizioni che potrebbero rendere l'espressione non definita, come radici di indice pari di argomenti negativi, denominatori nulli o argomenti di logaritmi non positivi. La soluzione è tipicamente un sottoinsieme di  $\mathbb{R}^2$  o  $\mathbb{R}^3$ , la cui rappresentazione grafica risulta spesso essenziale per visualizzare le regioni di validità della funzione."

---

**Listato 1:** Esempio di testo generato da Meteor contenente un messaggio cifrato. Il messaggio è in formato Markdown, il principale stile di formattazione degli LLM moderni.

In questo testo, i token non sono stati scelti tramite un campionamento casuale (*random sampling*) sulla testa della distribuzione, ma tramite la codifica dei bit del messaggio segreto. Per ogni passo, *Meteor* ha generato la distribuzione di probabilità tramite il modello linguistico, ha rimosso la coda della distribuzione sotto una soglia prefissata e ha ordinato i *logits* ottenuti. Gli indici, distribuiti proporzionalmente alle probabilità dei token in un intervallo da 0 a  $2^p$  (dove  $p$  è la precisione), sono stati usati per selezionare il token i cui bit più significativi corrispondessero ai bit successivi del messaggio cifrato.

Se, ad esempio, i bit da codificare fossero 0101110101... e le distribuzioni di probabilità dei token disponibili coprissero gli intervalli 0001..., 0100..., 0110... e 1010..., *Meteor* selezionerebbe il token associato all'intervallo [0100..., 0110...), codificando i primi due bit (01). Al passaggio successivo, il token selezionato entra a far parte del contesto del modello e i bit codificati vengono rimossi dalla testa del messaggio segreto, procedendo iterativamente fino al completamento o al raggiungimento dei limiti impostati.

Il testo risultante, coerente e naturale, transita sulla rete domestica senza destare sospetti. Bob, ricevendo il messaggio, utilizza lo stesso modello e la medesima chiave per estrarre i bit e decifrare la comunicazione, comprendendo la richiesta di Alice. Grazie a *Meteor*, Alice è riuscita a eludere la sorveglianza di Eve, rendendo la comunicazione indistinguibile da un comune output di un modello linguistico (e magari è riuscita anche ad ottenere la risposta al problema di analisi che stava studiando, ma questo non dipende da *Meteor*).

## 2.2 VQ-GAN: GENERAZIONE DI IMMAGINI AD ALTA RISOLUZIONE

### 2.2.1 *Contesto e motivazioni*

L'introduzione delle architetture GAN (Generative Adversarial Networks) ha rappresentato una svolta nel campo della generazione di immagini, consentendo la creazione di contenuti visivi di qualità, fino ad una dimensionalità massima (coerente) di  $256 \times 256$  pixel. Tuttavia, le GAN tradizionali hanno mostrato limitazioni significative nella generazione di immagini ad alta risoluzione, principalmente a causa della difficoltà nel mantenere la coerenza globale dell'immagine. In questo contesto, le VQ-GAN sono emerse come una soluzione innovativa, combinando i vantaggi delle GAN con un approccio di quantizzazione vettoriale che consente di generare immagini ad alta risoluzione in modo più efficiente. Gli indici derivanti dal processo di quantizzazione possono essere trattati come token, analogamente a quanto avviene nei modelli linguistici, e quindi essere processati da un transformer o da un'architettura affine, come GPT-2, il quale permette di mantenere consistenza nelle relazioni a lunga distanza, superando così le limitazioni delle GAN tradizionali e introducendo coerenza non solo a livello sintattico – capacità derivante dall'utilizzo di layer convolutivi – ma anche a livello semantico. Le VQ-GAN, grazie alla loro capacità di apprendere rappresentazioni discrete e compatte delle immagini, hanno dimostrato di essere particolarmente efficaci nella generazione di contenuti visivi complessi e dettagliati, rendendole, per un periodo, la scelta preferita per modelli destinati alla generazione di immagini ad alta risoluzione. Il successo dei Diffusion Transformer, che hanno superato le VQ-GAN in termini di qualità e coerenza delle immagini generate, ha portato a un declino dell'utilizzo di queste ultime. Tuttavia, le VQ-GAN rimangono una pietra miliare nello sviluppo della generazione di immagini ad alta risoluzione e continuano a essere oggetto di studio e applicazione in vari contesti, inclusa la steganografia generativa.

### 2.2.2 *Architettura del modello VQ-GAN*

La VQ-GAN (Vector Quantized – Generative Adversarial Network), introdotta in [32], rappresenta un'architettura ibrida che fonde i principi della quantizzazione vettoriale – mutuati dalla VQ-VAE (*Vector Quantized Variational Autoencoder*) [45] – con il paradigma avversario delle GAN [52] e, novità sostanziale, con la capacità di modellazione sequenziale dei

*Transformer* [47]. Benché l'articolo originale di Esser, Rombach e Ommer [32] formalizzi questo connubio tra VQ-GAN e Transformer, nel presente lavoro la VQ-GAN è impiegata specificamente come *modello di sintesi delle immagini* (generatore), utilizzando il suddetto transformer come equivalente pratico a GPT-2 per quanto illustrato per *Meteor* nella [sottosezione 2.1.2](#). Per questa ragione, la presente trattazione si concentra esclusivamente sul componente di sintesi visiva, rimandando al [Capitolo 3](#) la discussione del componente transformer e del suo ruolo nel processo steganografico.

### Formalizzazione del problema

Sia  $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$  un'immagine di input, con  $H$  e  $W$  rispettivamente altezza e larghezza in pixel e  $C$  il numero di canali colore (tipicamente  $C = 3$  per immagini RGB). L'obiettivo della VQ-GAN è duplice: 1) apprendere un *encoding* discreto e compatto dell'immagine, e 2) consentire la ricostruzione di immagini fedeli a partire da tale rappresentazione. Formalmente, si definisce un *codebook*  $\mathcal{Z} = \{\mathbf{z}_k\}_{k=1}^K \subset \mathbb{R}^{n_z}$ , ovvero un insieme finito di  $K$  vettori latenti di dimensionalità  $n_z$ , ciascuno dei quali rappresenta un *token visivo* elementare. La quantizzazione vettoriale consiste nel mappare ogni vettore dello spazio latente continuo  $\hat{\mathbf{z}} \in \mathbb{R}^{n_z}$  al suo corrispondente più vicino nel codebook secondo una metrica euclidea:

$$\mathbf{z}_q = \arg \min_{\mathbf{z}_k \in \mathcal{Z}} \|\hat{\mathbf{z}} - \mathbf{z}_k\|_2. \quad (1)$$

In questo modo, un'immagine  $\mathbf{x}$  viene trasformata in una griglia spaziale di indici  $\mathbf{s} \in \{1, \dots, K\}^{h \times w}$ , dove  $h = H/f$  e  $w = W/f$  sono le dimensioni spaziali del feature map dopo il downsampling operato dall'encoder, con  $f$  fattore di compressione spaziale. Questa griglia di indici discreti costituisce la *rappresentazione latente* dell'immagine e può essere linearizzata – tipicamente in ordine di scansione raster – per essere processata da un modello sequenziale come un Transformer.

### Componenti architetturali

L'architettura della VQ-GAN si compone di tre moduli fondamentali, ciascuno con una funzione specifica nel processo di codifica e ricostruzione dell'immagine:

**ENCODER (E).** L'encoder è una rete neurale convoluzionale profonda che trasforma l'immagine di input  $\mathbf{x}$  in una rappresentazione latente continua  $\hat{\mathbf{z}} = E(\mathbf{x}) \in \mathbb{R}^{h \times w \times n_z}$ . Attraverso una sequenza di strati

convoluzionali con *striding* e normalizzazione, l'encoder riduce progressivamente la risoluzione spaziale dell'immagine, producendo una mappa di feature densa che cattura le informazioni visive salienti a diversi livelli di astrazione.

**CODEBOOK ( $\mathcal{Z}$ ).** Il codebook costituisce il cuore del meccanismo di discretizzazione. Si tratta di un insieme apprendibile di  $K$  vettori latenti  $\mathbf{z}_k \in \mathbb{R}^{n_z}$ , tipicamente con  $K = 16384$  e  $n_z = 256$  nell'implementazione originale. Ogni vettore  $\hat{\mathbf{z}}_{ij}$  della mappa di feature continua prodotto dall'encoder viene sostituito dal vettore del codebook a esso più vicino in norma  $\|\cdot\|_2$ , generando la mappa di feature discretizzata  $\mathbf{z}_q$ . La scelta di  $K$  determina il trade-off tra fedeltà della ricostruzione e dimensione dello spazio latente discreto: un codebook più ricco consente di rappresentare una maggiore varietà di pattern visivi, ma aumenta la cardinalità del vocabolario su cui un eventuale modello generativo successivo (ad esempio un Transformer) dovrà apprendere la distribuzione condizionata.

**DECODER O GENERATORE ( $G$ ).** Il decoder, anch'esso implementato come rete convoluzionale profonda – spesso indicato come *generatore* nel lessico delle GAN – riceve in input la mappa di feature discretizzata  $\mathbf{z}_q$  e produce l'immagine ricostruita  $\tilde{\mathbf{x}} = G(\mathbf{z}_q) \in \mathbb{R}^{H \times W \times C}$ . La struttura del decoder è simmetrica a quella dell'encoder, utilizzando strati di *transposed convolution* (o *upsampling*) per riportare gradualmente la rappresentazione latente alla risoluzione originale.

A complemento della VQ-GAN, l'articolo originale [32] introduce una fase di modellazione successiva e indipendente: un **Transformer** autoregressivo (con architettura *decoder-only*, analoga a GPT-2) viene addestrato per apprendere la distribuzione di probabilità sugli indici discreti del codebook, trattando la sequenza linearizzata  $\mathbf{s} \in \{1, \dots, K\}^{h \cdot w}$  alla stregua di una sequenza di token. È in questa fase che si inserisce il meccanismo di campionamento steganografico descritto in precedenza: il messaggio segreto guida la selezione degli indici durante la generazione autoregressiva del Transformer, esattamente come avviene per i token testuali nel protocollo *Meteor* (sottosezione 2.1.2).

### *Discriminatore e funzione obiettivo*

Un elemento distintivo della VQ-GAN rispetto alla VQ-VAE originale è l'impiego di un discriminatore PatchGAN [43] (D), che opera a livello



di patch dell'immagine ( $70 \times 70$  pixel nell'implementazione standard) piuttosto che sull'intera immagine. Questo discriminatore produce una mappa di predizioni per ciascuna patch, consentendo di distinguere localmente tra regioni reali e sintetiche. La funzione obiettivo complessiva si articola in tre contributi:

$$\mathcal{L}_{\text{VQ-GAN}} = \mathcal{L}_{\text{VQ}} + \lambda_{\text{GAN}} \mathcal{L}_{\text{GAN}} + \lambda_{\text{perc}} \mathcal{L}_{\text{perc}}. \quad (2)$$

Il termine  $\mathcal{L}_{\text{VQ}}$  rappresenta la loss di quantizzazione vettoriale, composta a sua volta da due componenti:

$$\mathcal{L}_{\text{VQ}} = \|\text{sg}[\mathbb{E}(\mathbf{x})] - \mathbf{z}_q\|_2^2 + \beta \|\mathbb{E}(\mathbf{x}) - \text{sg}[\mathbf{z}_q]\|_2^2, \quad (3)$$

dove  $\text{sg}[\cdot]$  denota l'operatore *stop-gradient*, che impedisce al gradiente di propagarsi attraverso l'argomento specificato durante la retropropagazione. Il primo termine (*codebook loss*) addestra i vettori del codebook ad avvicinarsi agli output dell'encoder; il secondo termine (*commitment loss*), pesato da  $\beta$  (tipicamente  $\beta = 0.25$ ), vincola l'encoder a produrre rappresentazioni vicine ai vettori del codebook, prevenendo la divergenza dello spazio latente.

Il termine  $\mathcal{L}_{\text{GAN}}$  è la loss avversaria, definita come:

$$\mathcal{L}_{\text{GAN}}(G, D) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}_q \sim p_z} [\log(1 - D(G(\mathbf{z}_q)))]. \quad (4)$$

Questa componente introduce un gioco a somma zero tra il generatore  $G$  (che incorpora encoder e decoder) e il discriminatore  $D$ , dove  $G$  cerca di produrre immagini indistinguibili da quelle reali mentre  $D$  tenta di discriminarle. L'inclusione della loss avversaria è cruciale per migliorare la qualità percettiva delle ricostruzioni, in particolare per quanto riguarda texture e dettagli ad alta frequenza, ambito in cui le loss basate su distanza  $\|\cdot\|_2$  tipiche degli autoencoder variazionali tendono a produrre risultati eccessivamente lisci.

Il termine  $\mathcal{L}_{\text{perc}}$  utilizza una rete pre-addestrata (tipicamente VGG-16 [51]) per confrontare le feature percettive dell'immagine originale e di quella ricostruita:

$$\mathcal{L}_{\text{perc}} = \sum_l \|\phi_l(\mathbf{x}) - \phi_l(G(\mathbf{z}_q))\|_2^2, \quad (5)$$

dove  $\phi_l$  denota la mappa di attivazione al layer  $l$ -esimo della rete percettiva. Questa loss incoraggia la somiglianza semantica tra immagini originali e ricostruite, penalizzando deviazioni percettive che la sola loss ricostruttiva  $\|\cdot\|_2$  non è in grado di cogliere.

In fase di addestramento, i pesi  $\lambda_{\text{GAN}}$  e  $\lambda_{\text{perc}}$  vengono bilanciati dinamicamente tramite un meccanismo di *adaptive weight*: il contributo della loss avversaria viene regolato in modo tale che la sua magnitudine non domini quella della loss ricostruttiva, seguendo l'approccio descritto in [32]. Nello specifico,  $\lambda_{\text{GAN}}$  è definito come:

$$\lambda_{\text{GAN}} = \frac{\nabla_{G_L} [\mathcal{L}_{\text{VQ}}]}{\nabla_{G_L} [\mathcal{L}_{\text{GAN}}] + \delta'} \quad (6)$$

dove  $\nabla_{G_L}$  denota il gradiente rispetto all'ultimo layer del decoder e  $\delta$  è una costante di stabilizzazione ( $\delta = 10^{-6}$ ). Questo accorgimento impedisce che la loss avversaria prevalga sulla fedeltà ricostruttiva nelle fasi iniziali dell'addestramento.

#### *Motivazioni della scelta architetturale*

L'architettura VQ-GAN presenta diverse caratteristiche che la rendono particolarmente adatta al contesto della steganografia generativa visiva.

In primo luogo, la **natura discreta della rappresentazione latente** costituisce un ponte naturale con i modelli linguistici autoregressivi. Poiché la VQ-GAN produce una sequenza bidimensionale di indici discreti, questi possono essere linearizzati (ad esempio in ordine raster) e trattati alla stregua di token testuali da un modello di tipo Transformer. Questa proprietà consente di estendere il protocollo steganografico di *Meteor* (sottosezione 2.1.2) dal dominio linguistico a quello visivo: la codifica del messaggio segreto avviene durante la fase di generazione autoregressiva della sequenza di indici, esattamente come avviene per i token testuali. I codici discreti fungono da *vocabolario visivo* di cardinalità  $K$ , e la distribuzione condizionata  $P(z_t | z_{<t})$  appresa dal Transformer definisce lo spazio di campionamento all'interno del quale il messaggio può essere innestato.

In secondo luogo, la **compressione spaziale** operata dall'encoder riduce significativamente la lunghezza della sequenza di token da generare. Per un'immagine  $256 \times 256$ , con un fattore di compressione  $f = 16$ , la mappa latente ha dimensioni  $16 \times 16 = 256$  token. Questa riduzione della dimensionalità – che passa dai 65536 pixel originali a 256 token – rende computazionalmente trattabile la modellazione autoregressiva tramite Transformer, la cui complessità è quadratica nella lunghezza della sequenza ( $O(n^2)$ ). Senza tale compressione, la generazione diretta a livello di pixel sarebbe proibitiva per le risorse computazionali tipicamente disponibili.

In terzo luogo, la **qualità delle ricostruzioni** garantita dal training avversario e dalla loss percettiva assicura che le immagini prodotte a partire dai codici discreti siano visivamente plausibili e prive di artefatti evidenti. Il discriminatore PatchGAN opera localmente, rendendo il generatore particolarmente sensibile alla conservazione di texture e dettagli ad alta frequenza. Questo è fondamentale in contesti steganografici, dove anomalie visive anche minime potrebbero tradire la presenza di un messaggio occulto.

Infine, l'impiego di un **codebook appreso** – e non fissato euristicamente – consente alla rappresentazione latente di adattarsi alla distribuzione statistica del dominio visivo di interesse. I vettori del codebook vengono ottimizzati durante l'addestramento per minimizzare la loss di ricostruzione, garantendo che la discretizzazione preservi l'informazione visiva rilevante per la sintesi. Questo aspetto è particolarmente importante nel nostro contesto, poiché la codifica steganografica si inserisce in uno spazio latente specificamente progettato per bilanciare compattezza e potere espressivo.

### 2.3 LAVORI CORRELATI

L'evoluzione della steganografia generativa nel dominio visivo è strettamente legata al progresso delle tecniche di generazione multimediale, in particolare dei modelli *open-weights*. Nelle fasi iniziali, quando i modelli generativi presentavano limitazioni significative nella produzione di immagini sintatticamente e semanticamente coerenti, la ricerca si è prevalentemente orientata verso tecniche di *embedding* in immagini reali. L'obiettivo principale era minimizzare le anomalie statistiche e percettive introdotte dal processo di occultamento, perfezionando approcci consolidati come LSB (Least Significant Bit) e DCT (Discrete Cosine Transform) [42]. Parallelamente, sono state sviluppate varianti basate su architetture convolutive profonde [38, 40] e, più recentemente, ottimizzazioni finalizzate a migliorare la robustezza e l'impercettibilità dell'embedding [10, 11, 15, 17].

Tuttavia, queste metodologie si sono rivelate intrinsecamente vulnerabili all'avanzare delle tecniche di steganalisi, nate e sviluppate proprio allo scopo di individuare, con crescente accuratezza, la presenza di messaggi occulti all'interno di contenuti multimediali [35]. La letteratura più recente conferma come la capacità di discriminazione statistica degli strumenti di steganalisi abbia reso progressivamente meno efficaci gli approcci basati sulla sola modifica di cover preesistenti [14].

Con l'aumento della capacità architeturale e parametrica dei modelli generativi, si è assistito a un fenomeno sociale rilevante: la diffusione naturale di contenuti multimediali sintetici nello scambio quotidiano tra utenti. Ciò ha aperto la strada a un nuovo vettore steganografico – le immagini generate artificialmente – eliminando la necessità di ricorrere a immagini reali o preesistenti per l'embedding delle informazioni. Da questa constatazione è emersa la possibilità di adottare un approccio *coverless*, in cui l'immagine di copertura non preesiste, ma viene generata contestualmente all'incorporazione del messaggio segreto [21, 46]. Tale paradigma ha suscitato un crescente interesse nella comunità scientifica, portando alla proposta di diverse architetture basate su meccanismi generativi che integrano la codifica dell'informazione direttamente nel processo di sintesi [5–7, 12, 13, 16, 19, 20, 24, 31, 36, 37].

Le prime architetture in grado di generare immagini di qualità crescente, fino a una risoluzione massima coerente di  $256 \times 256$  pixel, sono state le reti GAN (Generative Adversarial Networks), caratterizzate da un'elevata densità di strati convolutivi. Nelle fasi iniziali di adozione, si è tuttavia osservata una marcata inconsistenza nelle immagini prodotte, le quali presentavano anomalie sia a livello sintattico sia semantico. Con l'evoluzione verso architetture più complesse, quali le VQ-GAN (Vector Quantized GAN), l'impiego di immagini generate come vettore steganografico è stato esplorato sistematicamente. In questa fase, le strategie adottate possono essere ricondotte a due filoni principali. Il primo, di natura conservativa, ha esteso le tecniche classiche di embedding (LSB, DCT) al nuovo contesto generativo, senza modificare sostanzialmente il paradigma operativo. Il secondo, più innovativo e affine all'approccio proposto nel presente lavoro, ha invece evitato di impiegare un'immagine reale come *cover object*, utilizzandola piuttosto come base contestuale per la generazione di un'immagine *ad hoc*. In questo quadro, il messaggio segreto viene codificato direttamente durante il processo generativo, superando le limitazioni connesse alla necessità di minimizzare le anomalie introdotte dall'embedding *a posteriori*. L'immagine prodotta risulta così progettata *ex ante* per incorporare l'informazione occulta in modo naturale e coerente.

Le architetture sviluppate in questa prima fase erano concepite per trasformare un contesto visivo di partenza in un'immagine generata secondo due modalità principali: da un lato, la modificazione di determinati aspetti dell'immagine originale, con il conseguente aumento del rischio di incorrere in errori sintattici o semantici; dall'altro, la *ricalcatura* delle feature più significative – quali i contorni degli oggetti, ovvero

le regioni a maggiore contrasto – preservando la struttura portante del contenuto originario. Il ricorso a un contenuto multimediale preesistente come base per la generazione non è tuttavia scomparso con il declino delle architetture GAN, ma è stato riproposto anche con l’avvento di architetture più performanti, come i Diffusion Transformer.

Un aspetto critico accomuna la maggior parte di queste tecniche: nessuna di esse ha raggiunto un livello di sicurezza steganografica formalmente dimostrabile. La ragione risiede nell’impossibilità di garantire una decodifica del messaggio priva di errori. Ciò è dovuto al fatto che tali metodi non si fondano sulla costruzione omnicomprensiva dell’immagine, bensì sulla modifica localizzata di alcune sezioni – talvolta marginali, talvolta limitate a dettagli stilistici di ridotta evidenza percettiva, come texture o rumore di fondo. Questa caratteristica intrinseca impedisce di assicurare sia l’integrità del messaggio decodificato sia l’indistinguibilità statistica della cover generata.

Queste considerazioni hanno motivato l’esplorazione di *Meteor* [28] – un framework steganografico formalmente sicuro originariamente concepito per il dominio testuale – in un contesto di generazione multimediale. L’obiettivo è stato quello di verificare se i principi alla base di tale framework potessero risolvere, almeno parzialmente, le criticità emerse nell’implementazione di un sistema steganografico visivo dotato di garanzie formali di sicurezza.



---

## DEFINIZIONI E TERMINOLOGIA

---

Nel seguito sono presentati un insieme di termini e definizioni fondamentali per la comprensione dei concetti chiave trattati nella tesi. I termini sono organizzati per aree tematiche omogenee, procedendo dai fondamenti della comunicazione sicura fino agli aspetti più specialistici della generazione steganografica. Questo lessico tecnico costituisce il vocabolario essenziale per navigare efficacemente all'interno del discorso sulla steganografia generativa, sui modelli linguistici, sulle architetture di reti neurali e sui generatori di immagini.

### 3.1 CONCETTI FONDAZIONALI DELLA COMUNICAZIONE

**Plaintext:** informazione nella sua forma originaria e integralmente leggibile, antecedente a qualsiasi trasformazione crittografica o steganografica. Nel contesto della presente trattazione, il plaintext costituisce il dato in chiaro che si intende proteggere.

**Ciphertext:** risultato dell'applicazione di un algoritmo di cifratura a un plaintext. Tale rappresentazione, incomprensibile in assenza della corrispondente chiave di decifrazione, costituisce il prodotto finale del processo crittografico.

**Coverobject:** oggetto multimediale originale, nella sua forma inalterata, che funge da contenitore all'interno del quale si intende occultare un messaggio segreto.

**Stegoobject:** oggetto risultante dall'incorporazione del messaggio segreto all'interno del coverobject. La proprietà fondamentale che tale artefatto deve soddisfare è l'indistinguibilità percettiva e statistica dal coverobject originale. Nel caso di *Meteor* o delle VQ-GAN, il termine si riferisce all'oggetto **generato** che contiene l'informazione segreta codificata, che non è in senso stretto un "oggetto modificato" ma un "oggetto creato" ad hoc. In questo contesto, il termine "stegoobject" è usato in

modo estensivo per indicare qualsiasi output generato che contenga un messaggio segreto, indipendentemente dal fatto che sia stato ottenuto tramite embedding o generazione ex novo.

**Payload:** quantità di informazione segreta, espressa in bit, che può essere occultata all'interno di un coverobject senza comprometterne l'integrità statistica e percettiva.

### 3.2 DISCIPLINE DELLA COMUNICAZIONE SICURA

**Crittografia:** disciplina formale che si occupa di rendere non intelligibile un messaggio ad attori non autorizzati mediante l'applicazione di metodi di cifratura. La crittografia costituisce la tecnica preminente per garantire la confidenzialità dei dati nei sistemi informatici contemporanei. Tale disciplina si fonda, ad oggi, principalmente su principi matematici e "algoritmi tradizionali" atti a trasformare l'informazione in una forma cifrata, decifrabile esclusivamente da chi possiede la chiave corretta. Pur costituendo il fondamento dei sistemi comunicativi moderni, la crittografia è potenzialmente vulnerabile ad attacchi di forza bruta, a exploit basati su vulnerabilità specifiche degli algoritmi, a compromissioni delle chiavi e, in prospettiva, ad algoritmi di decriptazione quantistica. Quando implementata correttamente, la crittografia fornisce un livello di sicurezza elevato, rendendo computazionalmente proibitivo l'accesso non autorizzato ai dati. [8]

**Cifratura simmetrica:** paradigma crittografico in cui il medesimo segreto condiviso viene impiegato tanto per la cifratura quanto per la decifrazione del messaggio. Tale metodologia, generalmente caratterizzata da una maggiore efficienza computazionale rispetto alla cifratura asimmetrica, richiede che la chiave sia trasmessa in modo sicuro tra le parti coinvolte.

**Steganografia:** disciplina che si occupa di occultare l'esistenza stessa di un messaggio all'interno di un oggetto multimediale, rendendolo impercettibile a osservatori non autorizzati. La steganografia si configura come tecnica complementare alla crittografia: mentre quest'ultima protegge il contenuto informativo, la steganografia tutela la riservatezza dell'atto comunicativo. Storicamente impiegata per finalità di spionaggio e comunicazione clandestina, la steganografia ha trovato nuove applicazioni nell'era digitale, quali la protezione dei diritti d'autore, la comunicazione sicura e la salvaguardia della privacy in rete [34]. Le modalità di implementazione includono l'inserimento di dati occulti in



immagini, tracce audio o flussi video, nonché l'impiego di tecniche di codifica testuale. La steganografia è tuttavia vulnerabile a tecniche di rilevamento avanzate (steganalisi) che analizzano le proprietà statistiche degli oggetti multimediali per identificare anomalie indicative della presenza di dati nascosti.

**Steganalisi:** disciplina deputata al rilevamento di messaggi occulti all'interno di oggetti multimediali, avvalendosi di metodologie di analisi statistica e di apprendimento automatico. La steganalisi costituisce il corrispettivo avversario della steganografia, concentrandosi sull'individuazione dell'esistenza stessa del messaggio celato piuttosto che sulla sua protezione. Con l'avvento delle tecnologie digitali, la steganalisi ha raggiunto livelli di sofisticazione notevoli, impiegando algoritmi avanzati per l'esame delle caratteristiche statistiche degli oggetti multimediali. [14]

**Steganografia senza embedding (SwE, Steganography without Embedding):** paradigma steganografico in cui il messaggio segreto non viene incorporato (*embedded*) all'interno di un coverobject preesistente, bensì il coverobject stesso viene **generato ex novo** a partire dal messaggio segreto. In questo approccio, un modello generativo (quale un modello linguistico o un generatore di immagini) produce un artefatto multimediale – testo, immagine o audio – la cui struttura e contenuto sono determinati dai bit del messaggio da occultare. Poiché non esiste un coverobject originale di riferimento, la steganalisi classica basata sul confronto tra coverobject e stegoobject risulta inefficace. L'indistinguibilità è garantita dalla qualità del modello generativo, che deve produrre output statisticamente e percettivamente equivalenti a campioni naturali. Paradigmi come *Meteor* per rappresentano istanze di questo approccio.

### 3.3 FONDAMENTI DI MODELLAZIONE LINGUISTICA

**Modelli linguistici:** classe di modelli di intelligenza artificiale progettati per comprendere e generare testo in linguaggio naturale. Tali modelli vengono addestrati su ingenti quantità di dati testuali e impiegano tecniche di apprendimento automatico per apprendere le regolarità grammaticali, il significato delle parole e le relazioni sintattico-semantiche tra le frasi. I modelli linguistici trovano applicazione in sistemi quali chatbot, piattaforme di traduzione automatica, generatori di testo e sistemi di analisi delle opinioni. Essi possono presentare vulnerabilità riconducibili a bias nei dati di addestramento e a limitazioni nella comprensione del

contesto.

**Modelli di linguaggio di grandi dimensioni (LLM):** sottocategoria di modelli linguistici caratterizzata da un numero di parametri elevato, che consente di apprendere rappresentazioni più complesse del linguaggio naturale e di generare testo con maggiore coerenza e naturalezza. Gli LLM vengono addestrati su corpora testuali di dimensioni massive e si avvalgono di architetture avanzate, quali i Transformer, per catturare le dipendenze a lungo termine tra le unità linguistiche.

**Modelli generativi:** classe di modelli di intelligenza artificiale atti a generare nuovi dati che presentano caratteristiche analoghe a quelli impiegati in fase di addestramento. Attraverso tecniche di apprendimento automatico, tali modelli apprendono la distribuzione sottostante ai dati di training e producono nuovi campioni che seguono la medesima distribuzione. I modelli generativi trovano impiego in numerosi ambiti, inclusi la generazione testuale, la sintesi di immagini, la composizione musicale e la sintesi vocale.

### 3.4 COMPONENTI DEI MODELLI LINGUISTICI

**Token:** unità testuale elementare impiegata nei modelli linguistici, corrispondente a una parola, un carattere o una sottoporzione di parola. I token costituiscono l'unità fondamentale su cui operano i modelli linguistici, rappresentando le entità minime che il modello deve elaborare e generare. La tokenizzazione – il processo di suddivisione del testo in token – può essere effettuata secondo diverse modalità, quali la segmentazione per parole, per caratteri o per sottoparole (ad esempio mediante algoritmi *subword* come Byte Pair Encoding).

**Vocabolario:** insieme dei token che un modello linguistico è in grado di processare e generare. Il vocabolario viene definito durante la fase di addestramento e la sua dimensione può variare in funzione del modello e dell'applicazione. Un vocabolario più esteso consente al modello di coprire una gamma più ampia di espressioni linguistiche, ma comporta un aumento della complessità computazionale.

**Logits:** valori numerici prodotti dall'ultimo strato di un modello generativo prima dell'applicazione della funzione di attivazione (tipicamente la softmax) per ottenere una distribuzione di probabilità sui token successivi. I logits rappresentano le preferenze del modello per ciascun token del vocabolario, dove valori più elevati indicano una maggiore propensione alla selezione.

**Softmax:** funzione di attivazione impiegata nei modelli linguistici per trasformare il vettore dei logits in una distribuzione di probabilità sull'insieme dei token del vocabolario. Formalmente,

$$\text{softmax}(x_i) = e^{x_i} / \sum_j e^{x_j},$$

dove  $x_i$  rappresenta il logit corrispondente al token  $i$ -esimo e la somma al denominatore è estesa all'intero vocabolario.

### 3.5 MECCANISMI DI GENERAZIONE E CONTROLLO

**Prompt:** sequenza testuale iniziale fornita in input a un modello linguistico per orientare e condizionare la generazione del testo successivo, definendo il contesto semantico e tematico dell'output prodotto.

**Contesto:** unione dei token del prompt e dei token generati fino a un dato punto temporale, che costituisce l'input su cui il modello linguistico basa la generazione del token successivo. Il contesto è dinamico e si evolve iterativamente con l'aggiunta di nuovi token, influenzando le distribuzioni di probabilità generate dal modello a ogni passo.

**Finestra di contesto:** limite massimo di token che un modello può considerare come input per la generazione del token successivo. La dimensione della finestra di contesto influisce sulla capacità del modello di mantenere coerenza e riferimenti a lungo termine all'interno del testo generato.

**Entropia:** nel contesto attuale, misura della variabilità o incertezza associata alla distribuzione di probabilità dei token successivi generata da un modello linguistico. Un'alta entropia indica una distribuzione più uniforme, con molte opzioni possibili per il token successivo, mentre una bassa entropia suggerisce una distribuzione concentrata su pochi token altamente probabili. L'entropia è un fattore cruciale nella steganografia generativa, poiché determina la capacità del sistema di codificare informazione senza compromettere la naturalezza del testo prodotto.

**Campionamento:** processo di selezione di un token successivo a partire da una distribuzione di probabilità generata da un modello. Le modalità di campionamento includono il campionamento casuale (*random sampling*), la selezione deterministica del token più probabile (*greedy decoding*), il campionamento casuale solo sui  $k$  token più probabili (*top-k sampling*), il campionamento casuale solo sui token con probabilità

sopra una certa soglia (*top-p sampling* o *nucleus sampling*) e il campionamento vincolato, come nel caso di *Meteor*, in cui la scelta del token è determinata dai bit del messaggio segreto.

### 3.6 ARCHITETTURE DI RETI NEURALI

**Transformer:** architettura di rete neurale basata sul meccanismo di attenzione (*self-attention*), ampiamente adottata nei modelli linguistici di grandi dimensioni per la capacità di catturare relazioni a lungo termine tra gli elementi di una sequenza, superando le limitazioni delle architetture ricorrenti nella modellazione di dipendenze distanti.

**Convolutional layer:** strato che applica operazioni di convoluzione per estrarre caratteristiche locali dai dati in ingresso, comunemente impiegato nelle reti neurali convoluzionali (CNN) per l'elaborazione di segnali strutturati su griglia, quali immagini e volumi multidimensionali.

**Recurrent layer:** strato che introduce connessioni ricorrenti per mantenere e propagare informazioni sullo stato precedente della sequenza, tipicamente utilizzato nelle reti neurali ricorrenti (RNN) per l'elaborazione di dati sequenziali, quali testo, serie temporali e segnali audio.

**Fully connected layer:** strato in cui ogni neurone è connesso a tutti i neuroni dello strato precedente, impiegato principalmente per la classificazione o la regressione dopo l'estrazione di caratteristiche da parte di strati convoluzionali o ricorrenti.

### 3.7 OPERAZIONI FONDAMENTALI DELLE RETI NEURALI

**Feature map:** insieme di attivazioni prodotte dall'applicazione di un filtro convoluzionale a un input, che cattura la presenza di determinate caratteristiche visive (bordi, texture, forme) in specifiche posizioni spaziali. Una feature map può essere pensata come una "mappa di risposta" che indica quanto ciascuna regione dell'input sia simile al pattern appreso dal filtro.

**Striding:** parametro che determina di quante posizioni si sposta un filtro convoluzionale (o un pool) a ogni passo durante la scansione dell'input. Un passo (o *stride*) maggiore di 1 riduce la risoluzione spaziale della feature map prodotta, operando di fatto un downsampling.

**Upsampling:** operazione che aumenta la risoluzione spaziale di una rappresentazione, tipicamente tramite interpolazione o attraverso strati

di *transposed convolution*, consentendo di riportare una mappa di feature compressa alla risoluzione originale.

**Downsampling:** operazione che riduce la risoluzione spaziale di una rappresentazione, ottenuta tipicamente tramite strati convoluzionali con *striding*  $> 1$  o tramite operazioni di *pooling* (come il *max pooling*). Questa compressione progressiva consente alla rete di apprendere feature via via più astratte e globali.

**Transposed convolution** (o *convoluzione trasposta*): operazione che effettua l'inverso "approssimato" di una convoluzione standard, aumentando la risoluzione spaziale dell'input. È comunemente impiegata nei decodificatori (o generatori) delle architetture convolutional per riportare una rappresentazione latente compressa alle dimensioni originali dell'immagine.

**Retropropagazione** (*backpropagation*): algoritmo fondamentale per l'addestramento delle reti neurali, che calcola il gradiente della funzione di loss rispetto a ciascun parametro del modello, propagando l'errore all'indietro attraverso gli strati della rete. Questo gradiente viene quindi utilizzato da algoritmi di ottimizzazione (come SGD o Adam) per aggiornare i pesi nella direzione che minimizza l'errore.

**Stop-gradient** (*sg[·]*): operatore formale che, durante la retropropagazione, blocca il flusso del gradiente attraverso l'argomento specificato. In pratica, ciò significa che il parametro contrassegnato viene trattato come una costante durante il calcolo delle derivate, impedendo che il gradiente della loss lo modifichi. È utilizzato, ad esempio, nelle loss di quantizzazione vettoriale per stabilizzare l'addestramento.

**Autoregressività:** proprietà di un modello generativo per cui la generazione di ciascun elemento della sequenza dipende esclusivamente dagli elementi precedenti (il "passato"). Formalmente, un modello autoregressivo fattorizza la probabilità congiunta di una sequenza come prodotto di probabilità condizionate:

$$P(x_1, x_2, \dots, x_n) = \prod_{t=1}^n P(x_t | x_{<t}).$$

Nel contesto di questa tesi, la generazione di token (testuali o visivi) avviene in modo autoregressivo: ogni nuovo token viene scelto in base a quelli già prodotti.

**Modello sequenziale:** modello specializzato nell’elaborazione o generazione di dati che presentano un ordinamento naturale (sequenze), quali token di testo, indici di codebook in ordine raster, o campioni audio. I Transformer, impiegati sia in *Meteor* che nella fase di modellazione delle VQ-GAN, sono un esempio di architettura sequenziale.

### 3.8 FUNZIONI OBIETTIVO E METRICHE DI ADDESTRAMENTO

**Funzione di loss** (o *funzione obiettivo*): funzione matematica che quantifica l’errore tra l’output prodotto da un modello e il target desiderato. Durante l’addestramento, il modello aggiorna i propri parametri per minimizzare il valore della loss, tipicamente tramite retropropagazione e discesa del gradiente.

**Loss ricostruttiva:** loss che misura la fedeltà con cui un modello ricostruisce i dati originali a partire da una rappresentazione compressa. Tipicamente utilizza la distanza  $\|\cdot\|_2$  (distanza euclidea o L2). Nel contesto della VQ-GAN, costituisce la componente principale dell’addestramento dell’autoencoder.

**Codebook loss:** componente della loss di quantizzazione vettoriale che addestra i vettori del codebook ad avvicinarsi agli output dell’encoder. Formalmente,

$$\mathcal{L}_{\text{codebook}} = \|\text{sg}[E(\mathbf{x})] - \mathbf{z}_q\|_2^2,$$

dove  $\text{sg}[\cdot]$  è l’operatore stop-gradient,  $E(\mathbf{x})$  è l’output dell’encoder e  $\mathbf{z}_q$  è il vettore quantizzato. L’operatore stop-gradient impedisce al gradiente di modificare l’encoder, concentrando l’aggiornamento esclusivamente sui vettori del codebook.

**Commitment loss:** componente della loss di quantizzazione vettoriale che vincola l’encoder a produrre rappresentazioni vicine ai vettori del codebook, prevenendo la divergenza dello spazio latente. Formalmente,

$$\mathcal{L}_{\text{commit}} = \beta \|E(\mathbf{x}) - \text{sg}[\mathbf{z}_q]\|_2^2,$$

dove  $\beta$  è un iperparametro (tipicamente  $\beta = 0.25$ ) che controlla il peso di questo termine. In questo caso, lo stop-gradient è applicato al vettore quantizzato, costringendo l’encoder a “allinearsi” al codebook.

**Loss avversaria** (*adversarial loss*): componente di loss introdotta dal gioco a somma zero tra generatore e discriminatore nelle GAN. Il generatore cerca di minimizzare questa loss producendo immagini che

ingannano il discriminatore, mentre il discriminatore cerca di massimizzarla distinguendo correttamente immagini reali da quelle generate. Formalmente:

$$\mathcal{L}_{\text{GAN}}(G, D) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}_q \sim p_z} [\log(1 - D(G(\mathbf{z}_q)))].$$

**Loss percettiva** (*perceptual loss*): componente di loss che confronta le rappresentazioni interne (feature map) di una rete pre-addestrata (tipicamente VGG-16) per l'immagine originale e quella ricostruita. Invece di confrontare i pixel direttamente, confronta le attivazioni a vari livelli di astrazione, penalizzando deviazioni semantiche e percettive. Formalmente:

$$\mathcal{L}_{\text{perc}} = \sum_l \|\phi_l(\mathbf{x}) - \phi_l(G(\mathbf{z}_q))\|_2^2,$$

dove  $\phi_l$  è l'attivazione al layer  $l$ -esimo della rete percettiva.

**Bilanciamento dinamico** (*adaptive weight*): meccanismo che regola automaticamente il peso dei diversi contributi della loss durante l'addestramento, impedendo che una componente (tipicamente la loss avversaria) domini sulle altre. Nella VQ-GAN, il peso  $\lambda_{\text{GAN}}$  della loss avversaria viene calcolato in modo che la magnitudine del suo gradiente non superi quella della loss ricostruttiva, seguendo la formula:

$$\lambda_{\text{GAN}} = \frac{\nabla_{G_L} [\mathcal{L}_{\text{VQ}}]}{\nabla_{G_L} [\mathcal{L}_{\text{GAN}}] + \delta'}$$

dove  $\nabla_{G_L}$  indica il gradiente rispetto all'ultimo layer del decoder e  $\delta$  è una costante di stabilizzazione ( $\delta \approx 10^{-6}$ ).

### 3.9 CLASSIFICATORI E RETI PER LA PERCEZIONE VISIVA

**PatchGAN**: discriminatore che opera non sull'intera immagine, ma su patch locali (tipicamente  $70 \times 70$  pixel), producendo una mappa di predizioni per ciascuna patch. Questo approccio costringe il generatore a concentrarsi sulla qualità locale delle texture e dei dettagli ad alta frequenza, piuttosto che sulla sola coerenza globale, risultando particolarmente efficace nel preservare la nitidezza delle immagini generate.

**VGG-16** [51]: architettura di rete neurale convolutiva profonda composta da 16 strati pesati (13 convoluzionali e 3 fully connected), pre-addestrata sul dataset ImageNet per la classificazione di immagini. Nel contesto della VQ-GAN, VGG-16 viene utilizzata come “estrattore di feature” fisso (non ulteriormente addestrato) per calcolare la loss percettiva, sfruttando la sua capacità di catturare rappresentazioni semantiche a diversi livelli di astrazione.

### 3.10 ARCHITETTURE PER LA GENERAZIONE VISUALE

**Generative Adversarial Network (GAN)**: architettura di rete neurale composta da due componenti distinti – un generatore e un discriminatore – che competono tra loro durante la fase di addestramento. Il generatore tenta di produrre dati sintetici indistinguibili da quelli reali, mentre il discriminatore si adopera per discriminare tra campioni autentici e campioni generati. Questo processo di apprendimento avversario conduce a un miglioramento progressivo della qualità dei dati sintetici prodotti.

**Vector Quantized Variational Autoencoder (VQ-VAE)** [45]: architettura precursore della VQ-GAN, che combina un autoencoder variazionale con la quantizzazione vettoriale del codice latente. A differenza della VQ-GAN, la VQ-VAE utilizza esclusivamente loss ricostruttiva (basata su distanza  $\|\cdot\|_2$ ) per l’addestramento, senza componente avversaria né loss percettiva. Questo comporta ricostruzioni di qualità inferiore, in particolare nelle texture e nei dettagli ad alta frequenza, che tendono a risultare eccessivamente lisci.

**Vector Quantized-Generative Adversarial Network (VQ-GAN)** [32]: architettura di rete neurale che integra i principi dei GAN con tecniche di quantizzazione vettoriale, impiegando un codebook discreto per rappresentare le caratteristiche latenti dei dati in ingresso. La VQ-GAN è progettata per migliorare la qualità e la stabilità del processo generativo, riducendo al contempo la complessità computazionale mediante la discretizzazione dello spazio latente. Rispetto alla VQ-VAE di cui costituisce un’evoluzione, la VQ-GAN introduce una loss avversaria (tramite un discriminatore PatchGAN) e una loss percettiva (tramite una rete VGG-16 pre-addestrata), che migliorano significativamente la nitidezza e la fedeltà visiva delle ricostruzioni.

**Codebook**: insieme strutturato di vettori latenti appresi durante l’addestramento di un modello basato su quantizzazione vettoriale, come la



VQ-GAN. Ciascun vettore del codebook costituisce un elemento atomico del dizionario di rappresentazioni discrete mediante il quale i dati originali vengono codificati e successivamente ricostruiti. La cardinalità del codebook (tipicamente  $K = 16384$ ) determina il trade-off tra fedeltà della ricostruzione e dimensione dello spazio latente discreto.

**Quantizzazione:** processo di discretizzazione di uno spazio di rappresentazione continuo (o discreto ma di dimensione eccessivamente elevata) in un insieme finito di valori atomici detti *token*. Tale procedura viene impiegata in architetture come la VQ-GAN per ridurre la complessità del modello e migliorare la stabilità del processo di addestramento, trasformando lo spazio latente continuo in un codebook finito di vettori discreti. La scelta del vettore del codebook più vicino a un dato vettore continuo avviene tipicamente tramite minimizzazione della distanza euclidea.

### 3.11 MISURE E GARANZIE NEI SISTEMI STEGANOGRAFICI

**Bitrate variabile:** strategia di codifica in cui la quantità di informazione incorporata in ciascun token può variare dinamicamente in funzione delle caratteristiche del contesto e della distribuzione di probabilità dei token successivi. In contesti a bassa entropia, il sistema può astenersi dalla codifica di informazione, mentre in scenari ad alta entropia può codificare un numero maggiore di bit per token, massimizzando il throughput informativo senza compromettere la naturalezza statistica del testo generato.

**Throughput informativo:** quantità di informazione segreta, misurata in bit, che può essere codificata e trasmessa attraverso un canale di comunicazione, quale un testo generato da un modello linguistico. Il throughput informativo è determinato da molteplici fattori, tra cui la capacità di codifica del modello sottostante, la distribuzione di probabilità dei token e la strategia di codifica adottata.

**Anomalia semantica:** incoerenza o incongruenza nel significato di un testo generato, riconducibile a selezioni di token non naturali o a errori nel processo generativo. Le anomalie semantiche costituiscono indicatori potenzialmente rivelatori per strumenti di steganalisi, in quanto segnalano una deviazione dal comportamento generativo naturale.

**Soglia di entropia minima:** valore critico dell'entropia al di sotto del quale un sistema di steganografia generativa, quale *Meteor*, si astiene

dalla codifica di informazione segreta. Tale soglia garantisce che l'inserimento di informazioni occulte avvenga esclusivamente in contesti in cui la distribuzione di probabilità dei token successivi presenta una varianza sufficiente, prevenendo la generazione di anomalie semantiche.

**Soglia di probabilità:** valore prefissato impiegato per troncare la distribuzione di probabilità dei token successivi prodotta da un modello linguistico, eliminando i token la cui probabilità risulta inferiore a tale soglia. Questo meccanismo, noto come troncamento della coda (*tail truncation*), circoscrive le scelte ai token maggiormente probabili, migliorando la coerenza e la naturalezza del testo generato.

**Troncamento della coda:** tecnica impiegata nei modelli linguistici per limitare l'insieme dei token candidati alla selezione a quelli la cui probabilità supera una soglia prefissata. Tale procedura migliora la coerenza e la naturalezza del testo generato, eliminando i token meno probabili che potrebbero introdurre anomalie semantiche o statistiche. Questa tecnica riveste particolare rilevanza in sistemi di steganografia generativa, come *Meteor*, in quanto contribuisce a garantire che il testo prodotto rimanga statisticamente indistinguibile da un testo naturale.

**Indistinguibilità statistica:** proprietà di un testo generato da un modello linguistico, quale *Meteor*, per cui le sue caratteristiche statistiche risultano indistinguibili da quelle di un testo naturale non steganografico. Tale proprietà è fondamentale per impedire a strumenti di steganalisi di rilevare la presenza di informazioni occulte basandosi su anomalie statistiche.

**Indistinguibilità percettiva:** proprietà di un testo generato da un modello linguistico, quale *Meteor*, per cui il suo significato e la sua coerenza risultano indistinguibili da quelli di un testo naturale non steganografico. Tale proprietà contribuisce a garantire che il testo prodotto non susciti sospetti basati su anomalie semantiche.

**Non-determinismo:** caratteristica di un sistema di generazione per cui l'output prodotto non è deterministico ma dipende da fattori probabilistici, tra cui la distribuzione di probabilità dei token successivi e la strategia di codifica steganografica adottata. Il non-determinismo contribuisce a garantire l'indistinguibilità statistica e percettiva del messaggio generato. Nel sistema implementato in questa tesi, il non-determinismo ha tuttavia rappresentato una criticità: l'architettura VQ-GAN impiega un codificatore (*Encoder*) e un decodificatore (*Decoder*) che, essendo funzioni statistiche apprese e non inversi esatti, introducono una componente d'errore nel processo di ricostruzione. Questa mancata biettività

ha prodotto inconsistenze tra la codifica e la decodifica di alcune patch dell'immagine.



---

## ARCHITETTURA DEL SISTEMA PROPOSTO

---

### 4.1 VISIONE D'INSIEME

Nei capitoli precedenti sono stati presentati i fondamenti teorici necessari alla comprensione del lavoro, con particolare attenzione al protocollo *Meteor* ([sezione 2.1](#)) e all'architettura VQ-GAN ([sezione 2.2](#)). Il presente capitolo descrive in dettaglio gli aspetti algoritmici e teorici del sistema steganografico proposto, che estende il principio di *Steganography without Embedding* (**SwE**) di *Meteor* dal dominio testuale a quello visivo.

#### 4.1.1 Scelta del modello di generazione delle immagini

Per la componente generativa è stato selezionato il checkpoint pre-addestrato **S-FLCKR** ( $f = 16$ ) della VQ-GAN, reso disponibile dagli autori di *Taming Transformers* [32] e addestrato sul dataset *outdoor* di paesaggi naturali. La scelta di un modello specializzato su paesaggi deriva da una considerazione percettiva: le immagini di paesaggi presentano una naturale variabilità cromatica e strutturale (cielo, alberi, acqua, montagne) che rende le eventuali distorsioni meno percepibili a un osservatore umano, a differenza di modelli addestrati su categorie semanticamente più rigide (volti, oggetti) che tendono a produrre artefatti più facilmente individuabili.

### 4.2 IDEA FONDANTE

L'idea fondante del sistema è la seguente: così come *Meteor* utilizza un modello linguistico autoregressivo (GPT-2) per generare testo in cui il messaggio segreto determina la selezione dei token, allo stesso modo è possibile impiegare il *Transformer* associato a una VQ-GAN per generare una sequenza di codici discreti (token visivi) in cui il messaggio segreto

guida il campionamento. La sequenza di indici così prodotta viene quindi decodificata dal decoder VQ-GAN per ottenere l'immagine finale, detta *stego-image*. La scelta del modello VQ-GAN deriva quindi direttamente dall'implementazione del Codebook  $\mathcal{K}$  di token discreti, che consente di applicare il meccanismo di selezione steganografica praticato da *Meteor*, adattandolo così direttamente al dominio visivo tramite l'utilizzo del Transformer autoregressivo.

Il sistema implementato si articola in due fasi principali:

**ENCODING STEGANOGRAFICO:** il mittente, dato un messaggio segreto, genera un'immagine contenente il messaggio cifrato nascosto nel processo di campionamento dei token visivi.

**DECODING STEGANOGRAFICO:** il destinatario, in possesso dello stesso modello VQ-GAN e della stessa chiave condivisa, estrae il messaggio segreto dall'immagine ricevuta ricostruendo le distribuzioni di probabilità e recuperando i bit che hanno determinato la selezione dei token.

Un aspetto cruciale del sistema è la sua *non intrusività*: l'insieme dei pesi pre-addestrati della VQ-GAN non viene né rifinito (o *fine-tuned*) né modificato in alcun modo. Il messaggio segreto viene codificato esclusivamente durante la fase di inferenza del Transformer, orientando il campionamento dei token visivi senza alterare le distribuzioni di probabilità apprese dal modello. Questa proprietà garantisce che l'immagine generata sia statisticamente indistinguibile da una immagine prodotta dal medesimo modello in assenza di messaggio nascosto, realizzando di fatto una steganografia perfetta nel senso della definizione di *perfectly secure steganography* introdotta in [28].

### 4.3 FLUSSO OPERATIVO DEL SISTEMA

Il funzionamento complessivo del sistema può essere descritto nei seguenti passi:

1. **Preparazione del contesto:** Viene selezionata un'immagine di contesto dal dataset di riferimento. Questa viene processata dall'encoder VQ-GAN per ottenere una sequenza di indici discreti del codebook. Le prime  $R$  righe della griglia di indici (dove  $R$  è determinato dal parametro `context_fraction`) fungono da contesto iniziale non steganografico e vengono preservate nell'immagine finale.

2. **Inizializzazione della sequenza:** Gli indici corrispondenti alle prime  $R$  righe vengono copiati in un tensore che conterrà progressivamente i token generati. Le restanti posizioni vengono azzerate.
3. **Generazione autoregressiva vincolata:** Per ogni posizione della griglia a partire dalla riga  $R + 1$ , il Transformer produce una distribuzione di probabilità  $P(z_t | z_{<t})$  su tutti i  $K$  codici del codebook, condizionata dal contesto locale costituito da una finestra di  $p \times p$  token visivi. Il messaggio cifrato determina la selezione del token tramite un meccanismo di *arithmetic coding*.
4. **Ricostruzione dell'immagine:** Una volta completata la generazione di tutti i token della griglia, la sequenza di indici viene decodificata dal decoder VQ-GAN per produrre l'immagine finale.
5. **Estrazione del messaggio:** Il destinatario, in possesso dello stesso modello, della stessa informazione di contesto e della chiave condivisa, riesegue il processo di generazione delle distribuzioni e, confrontando i token effettivamente presenti nell'immagine ricevuta con le distribuzioni attese, estrae i bit del messaggio originale. Si tenga presente che, ai fini del sistema implementato, è stato sufficiente utilizzare il medesimo insieme di pesi e una chiave privata, da cui è quindi possibile ricostruire lo stesso contesto iniziale utilizzando un dataset condiviso, senza necessità di condividere l'immagine di contesto originale.

#### 4.4 RAPPRESENTAZIONE DELLE IMMAGINI E GRIGLIA DI PATCH

Un'immagine  $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$  viene processata dall'encoder VQ-GAN per produrre una mappa di feature latente  $\hat{\mathbf{z}} \in \mathbb{R}^{h \times w \times n_z}$ , dove  $h = H/f$  e  $w = W/f$  sono le dimensioni della griglia di patch e  $f$  è il fattore di compressione spaziale (nell'implementazione utilizzata,  $f = 16$ ). Ciascuna posizione  $(i, j)$  della mappa latente viene quantizzata al vettore del codebook più vicino (tramite l'utilizzo della norma euclidea  $\|\cdot\|_2$ ), producendo un indice discreto  $s_{ij} \in \mathcal{K}$ , dove  $\mathcal{K} := \{1, \dots, K\}$ . L'intera immagine viene quindi rappresentata come una matrice di indici  $\mathbf{S} \in \mathcal{K}^{h \times w}$ .

Nel sistema proposto, il Transformer opera su una finestra locale di dimensione  $p \times p$  token, dove  $p$  è la dimensione della finestra di contesto in unità di token. Per ciascuna posizione  $(r, c)$  della griglia, il contesto è costruito concatenando due porzioni:

- Gli indici della patch di riferimento (*reference patch*), estratti dall'immagine di contesto;
- Gli indici correntemente generati (*building patch*), che includono i token già selezionati per le posizioni precedenti.

#### 4.5 ALGORITMO DI ENCODING STEGANOGRAFICO

L'algoritmo di encoding steganografico costituisce il cuore del sistema. Il suo obiettivo è trasformare un messaggio testuale in un'immagine, codificando i bit del messaggio nella selezione dei token visivi durante la generazione autoregressiva.

##### 4.5.1 Selezione steganografica del token

Il meccanismo di selezione del token ([Algoritmo 1](#)) implementa il principio alla base di *Meteor*, adattato al dominio visivo: dato il contesto delle patch precedenti, il Transformer produce una distribuzione di probabilità su tutti i  $K$  codici del codebook; il messaggio segreto determina quale token selezionare all'interno di questa distribuzione, e il numero di bit effettivamente codificati corrisponde al prefisso binario comune degli estremi dell'intervallo selezionato. Per preservare la qualità visiva, le code della distribuzione vengono troncate conservando solo i token con probabilità  $p_i \geq 1/K$ , e le probabilità residue vengono normalizzate e quantizzate nello spazio  $\{0, \dots, K - 1\}$ .

##### 4.5.2 Codifica vincolata di un singolo token

Un aspetto critico del dominio visivo è la non-reversibilità dell'encoder VQ-GAN: a differenza di un modello linguistico, dove il token selezionato compare esattamente nell'output, per la VQ-GAN l'indice ottenuto ricodificando l'immagine generata tramite l'encoder VQ-GAN può differire da quello originariamente scelto. Per garantire la corretta decodifica steganografica, ogni token selezionato viene verificato decodificando l'immagine parziale tramite il decoder VQ-GAN e ricodificandola tramite l'encoder VQ-GAN; in caso di discordanza, la selezione viene ripetuta fino a un numero massimo di tentativi.



**Algoritmo 1** Selezione steganografica del token visivo

**Require:** Contesto  $\mathbf{c}$ , messaggio binario  $\mathbf{b}$ , dimensione codebook  $K$ , precisione  $D$ , soglia  $T$

**Ensure:** Indice selezionato  $i^*$ , bit codificati  $n$ , estremi intervallo  $L, R$ , bit residui  $\mathbf{b}'$

- 1:  $\mathbf{p} \leftarrow \text{softmax}(\text{Transformer}(\mathbf{c}))$
- 2:  $\mathcal{J} \leftarrow \{i : p_i \geq 1/K\}$   $\triangleright$  selezione dei token con probabilità significativa
- 3:  $\mathcal{J} \leftarrow \mathcal{J}[0 : \min(|\mathcal{J}|, T)]$   $\triangleright$  troncamento al top- $T$
- 4:  $\hat{p}_i \leftarrow \lfloor (p_i / \sum_{j \in \mathcal{J}} p_j) \cdot K \rfloor$  per  $i \in \mathcal{J}$
- 5:  $C_i \leftarrow \sum_{j=1}^i \hat{p}_j$  per  $i = 1, \dots, |\mathcal{J}|$   $\triangleright$  accumulatore delle probabilità quantizzate
- 6:  $m \leftarrow \text{bin2dec}(\mathbf{b}[1 : D])$   $\triangleright$  primi  $D$  bit convertiti in intero
- 7:  $i^* \leftarrow \min\{i : C_i > m\}$   $\triangleright$  selezione tramite arithmetic decoding
- 8:  $L \leftarrow C_{i^*-1}$  (o 0 se  $i^* = 1$ ),  $R \leftarrow C_{i^*}$   $\triangleright$  intervallo selezionato
- 9:  $n \leftarrow \text{conta\_bit\_comuni}(\text{dec2bin}(L, D), \text{dec2bin}(R - 1, D))$
- 10:  $\mathbf{b}' \leftarrow \text{dec2bin}(L, D)[1 : n]$
- 11: **return**  $i^*, n, L, R, \mathbf{b}'$

**Algoritmo 2** Codifica vincolata di un singolo token

**Require:** Matrice di riferimento  $\mathbf{R}$ , matrice in costruzione  $\mathbf{B}$ , posizione  $(r, c)$ , dimensioni griglia  $(h, w)$ , bitstream  $\mathbf{b}$ , flag di campionamento casuale  $q$

**Ensure:** Posizione successiva  $(r', c')$ , indice  $i^*$ , bit codificati  $n$ , bitstream aggiornato  $\mathbf{b}'$

- 1:  $\mathbf{c} \leftarrow \text{costruisci\_contesto}(\mathbf{R}, \mathbf{B}, r, c, (h, w))$
- 2: **repeat**
- 3:    $(i^*, n, \_, \_, \mathbf{b}') \leftarrow \text{seleziona\_token}(\mathbf{c}, \mathbf{b})$
- 4:    $\mathbf{B}[r, c] \leftarrow i^*$
- 5:    $\hat{\mathbf{i}} \leftarrow [\text{Encoder}_{VQ}(\text{Decoder}_{VQ}(\mathbf{B}))]_{r, c}$
- 6: **until**  $q = \text{vero}$  **or**  $\hat{\mathbf{i}} = i^*$  **or**  $\text{tentativi} > 100$
- 7: **return**  $\text{pos\_successiva}(r, c, (h, w)), i^*, n, \mathbf{b}'$

### 4.5.3 Flusso principale di encoding steganografico

**Algoritmo 3** coordina l'intero processo di encoding steganografico a livello di immagine: dopo aver preparato il contesto iniziale e i tensori di riferimento, itera sulle posizioni della griglia selezionando token guidati dal messaggio fino all'esaurimento dei bit, quindi completa le posizioni rimanenti con campionamento casuale per preservare la coerenza visiva.

---

#### **Algoritmo 3** Codifica steganografica del messaggio in immagine

---

**Require:** Messaggio  $M$ , modello VQ-GAN, dataset  $D$ , frazione di contesto  $f$

**Ensure:** Immagine  $\mathbf{x}$ , sequenza indici  $\mathbf{s}$ , bit residui  $\mathbf{b}$

```

1:  $(\mathbf{R}, \mathbf{B}, r_0, (h, w)) \leftarrow \text{prepara\_contesto}(\text{modello}, D, f)$ 
2:  $\mathbf{b} \leftarrow \text{testo2bin}(M)$   $\triangleright$  conversione del messaggio in sequenza binaria
3:  $(r, c) \leftarrow (r_0, 0)$ 
4: while  $\mathbf{b} \neq \emptyset$  and  $r < h$  do
5:    $((r, c), i^*, n, \mathbf{b}') \leftarrow \text{codifica\_patch}(\mathbf{R}, \mathbf{B}, r, c, (h, w), \mathbf{b}[1 : D])$ 
6:    $\mathbf{s} \leftarrow \mathbf{s} \cup \{i^*\}; \quad \mathbf{b} \leftarrow \mathbf{b}[n + 1 : ]$ 
7: end while
8: while  $r < h$  do
9:    $((r, c), i^*, \dots) \leftarrow \text{codifica\_patch}(\mathbf{R}, \mathbf{B}, r, c, (h, w); q = \text{vero})$ 
10:   $\mathbf{s} \leftarrow \mathbf{s} \cup \{i^*\}$ 
11: end while
12:  $\mathbf{x} \leftarrow \text{Decoder}_{\text{VQ}}(\mathbf{B})$ 
13: return  $\mathbf{x}, \mathbf{s}, \mathbf{b}$ 
```

---

## 4.6 ALGORITMO DI DECODING STEGANOGRAFICO

L'algoritmo di decoding steganografico costituisce il processo inverso: data un'immagine ricevuta, il destinatario estrae il messaggio nascosto. La decodifica steganografica è possibile perché mittente e destinatario condividono lo stesso modello VQ-GAN, la stessa informazione di contesto (tipicamente un seme pseudo-casuale) e la stessa chiave simmetrica per la cifratura del messaggio.

### 4.6.1 Estrazione dei bit da un token

Il meccanismo di estrazione (**Algoritmo 4**) è simmetrico rispetto all'encoding steganografico: per ciascuna posizione della griglia, la distribu-

zione di probabilità viene ricostruita applicando lo stesso troncamento e quantizzazione utilizzati in fase di encoding steganografico; il token effettivamente presente nell'immagine identifica l'intervallo  $[C_{j-1}, C_j)$  corrispondente, e i bit vengono recuperati dal prefisso binario comune degli estremi dell'intervallo.

---

**Algoritmo 4** Decodifica steganografica dei bit da un token visivo

---

**Require:** Contesto  $\mathbf{c}$ , token ricevuto  $z^*$ , dimensione codebook  $K$ , precisione  $D$ , soglia  $T$

**Ensure:** Bit decodificati  $\mathbf{b}'$ , estremi intervallo  $L, R$

- 1:  $\mathbf{p} \leftarrow \text{softmax}(\text{Transformer}(\mathbf{c}))$
  - 2:  $\mathcal{J} \leftarrow \{i : p_i \geq 1/K\} \triangleright$  stesso troncamento dell'encoding steganografico
  - 3:  $\mathcal{J} \leftarrow \mathcal{J}[0 : \min(|\mathcal{J}|, T)]$
  - 4:  $\hat{p}_i \leftarrow \lfloor (p_i / \sum_{j \in \mathcal{J}} p_j) \cdot K \rfloor$  per  $i \in \mathcal{J}$
  - 5:  $C_i \leftarrow \sum_{j=1}^i \hat{p}_j$  per  $i = 1, \dots, |\mathcal{J}|$
  - 6:  $j \leftarrow \text{cerca\_indice}(z^*, \mathcal{J}) \triangleright$  posizione del token ricevuto nell'insieme selezionato
  - 7:  $L \leftarrow C_{j-1}$  (o 0 se  $j = 0$ ),  $R \leftarrow C_j$
  - 8:  $n \leftarrow \text{conta\_bit\_comuni}(\text{dec2bin}(L, D), \text{dec2bin}(R - 1, D))$
  - 9:  $\mathbf{b}' \leftarrow \text{dec2bin}(L, D)[1 : n]$
  - 10: **return**  $\mathbf{b}'$ ,  $L$ ,  $R$
- 

#### 4.6.2 Flusso principale di decoding steganografico

**Algoritmo 5** illustra il processo completo: il destinatario ricostruisce le stesse distribuzioni di probabilità percorrendo la griglia nell'ordine raster, estrae i bit da ciascun token e ricompone il messaggio originale.

### 4.7 GESTIONE DEL CONTESTO E SLIDING WINDOW

Nel dominio visivo, a differenza di *Meteor* dove il contesto è costituito dall'intera sequenza testuale, il Transformer della VQ-GAN opera su una finestra locale  $p \times p$  centrata sulla posizione corrente. Per ciascuna posizione  $(r, c)$  della griglia, il contesto viene costruito concatenando due finestre della stessa dimensione: una dagli indici di riferimento  $\mathbf{R}$  (immagine di contesto originale) e una dagli indici in costruzione  $\mathbf{B}$  (token già selezionati). La sequenza risultante di  $2 \times p \times p$  token viene appiattita e fornita come input al Transformer.

**Algoritmo 5** Decodifica steganografica del messaggio dall'immagine

**Require:** Immagine stego  $\mathbf{x}$ , modello VQ-GAN, contesto  $D$ , frazione di contesto  $f$

**Ensure:** Messaggio decodificato  $M'$ , bit  $\mathbf{b}'$

---

```

1:  $(\mathbf{R}, \mathbf{B}_d, r_0, (h, w)) \leftarrow \text{prepara\_contesto}(\text{modello}, D, f)$ 
2:  $\mathbf{S} \leftarrow \text{Encoder}_{VQ}(\mathbf{x})$   $\triangleright$  mappa degli indici nell'immagine ricevuta
   tramite encoding VQ-GAN
3:  $(r, c) \leftarrow (r_0, 0)$ ;  $\mathbf{b}' \leftarrow \emptyset$ 
4: while  $r < h$  do
5:    $\mathbf{c} \leftarrow \text{costruisci\_contesto}(\mathbf{R}, \mathbf{B}_d, r, c, (h, w))$ 
6:    $\mathbf{z}^* \leftarrow \mathbf{S}[r, c]$   $\triangleright$  token effettivo nell'immagine stego
7:    $\mathbf{b}' \leftarrow \mathbf{b}' + \text{decodifica\_token}(\mathbf{c}, \mathbf{z}^*)$ 
8:    $\mathbf{B}_d[r, c] \leftarrow \mathbf{z}^*$ 
9:    $(r, c) \leftarrow \text{pos\_successiva}(r, c, (h, w))$ 
10: end while
11:  $M' \leftarrow \text{bin2testo}(\mathbf{b}')$ 
12: return  $M', \mathbf{b}'$ 

```

---

Il parametro `context_fraction` ( $f$ ) determina quante righe iniziali della griglia vengono preservate dall'immagine di contesto originale. Con il valore predefinito  $f = 0.1$ , per una griglia  $h \times w$  (nell'implementazione  $16 \times 16$ ) si preservano circa 25 token di contesto. Un valore troppo basso riduce la qualità delle distribuzioni di probabilità; un valore troppo alto riduce la capacità steganografica complessiva.

#### 4.8 LIMITAZIONI TEORICHE

Il sistema presenta alcune limitazioni intrinseche che è importante esplicitare.

##### 4.8.1 Non-determinismo della VQ-GAN

L'encoder e il decoder della VQ-GAN non costituiscono funzioni inverse esatte:

$$\text{Decoder}_{VQ}(\text{Encoder}_{VQ}(\mathbf{x})) \neq \mathbf{x}, \quad \text{per alcuni } \mathbf{x} \in \mathbb{R}^{H \times W \times C}$$

Questa asimmetria implica che il token selezionato durante l'encoding steganografico potrebbe non corrispondere a quello ottenuto ricodificando l'immagine tramite l'encoder VQ-GAN. Il meccanismo di verifica

descritto nell'[Algoritmo 2](#) mitiga il problema, ma in casi estremi (quando nessun token supera la verifica entro il numero massimo di tentativi) la scelta viene forzata, con possibile perdita di bit. Inoltre il meccanismo di decodifica VQ-GAN non è attuabile per singole patch o per immagini parziali, ma solo a livello di immagine completa in quanto è una rete neurale che opera su vettori completi di indici come input e non è possibile isolare il contributo di un singolo token alla ricostruzione visiva.

#### 4.8.2 *Capacità steganografica*

Per un'immagine  $256 \times 256$  con  $f = 16$ , la griglia ha  $h \times w = 16 \times 16 = 256$  token. Sottraendo 16-32 token di contesto, restano circa 224-240 token codificanti. Con precisione  $D = 10$  bit, idealmente si potrebbero codificare fino a 10 bit per token; in pratica, il numero effettivo dipende dall'entropia della distribuzione e si attesta in media tra 3 e 7 bit per token, per una capacità complessiva di circa 140 caratteri ASCII per immagine.



---

## IMPLEMENTAZIONE DEL SISTEMA

---

Il presente capitolo descrive l'implementazione pratica del sistema steganografico presentato nel [Capitolo 4](#). Mentre il capitolo precedente si è concentrato sugli aspetti algoritmici e teorici — in particolare gli [Algoritmo 1](#), [2](#) e [3](#) per l'encoding steganografico, e gli [Algoritmo 4](#) e [5](#) per il decoding steganografico — qui vengono illustrati i dettagli realizzativi: l'architettura software, l'organizzazione del codice in classi Python, e le procedure per la generazione e il salvataggio delle immagini.

### 5.1 ARCHITETTURA SOFTWARE

#### 5.1.1 *Scelta del linguaggio di programmazione e del framework*

L'intero sistema è stato sviluppato in Python con il framework *PyTorch*. Sia l'implementazione originale di VQ-GAN [32] sia quella di *Meteor* [28] sono sviluppate in questo ecosistema, la cui adozione consente di riutilizzare direttamente i pesi pre-addestrati dei modelli esistenti e di garantire la riproducibilità dei risultati. Per motivi strettamente pratici non sono state prese in considerazione alternative come TensorFlow o JAX, che avrebbero richiesto una riscrittura completa del codice e una conversione dei modelli, con conseguente rischio di introdurre discrepanze nei risultati, senza una reale giustificazione in termini di prestazioni o funzionalità specifiche.

#### 5.1.2 *Moduli principali*

Il sistema implementato si compone di tre moduli principali, ciascuno corrispondente a un file sorgente:

`vqgan.py`: interfaccia con il modello VQ-GAN pre-addestrato. Espone le funzioni `encode_to_z()` e `decode_to_img()`, che astraggono

rispettivamente l'encoding VQ-GAN di un'immagine nella corrispondente griglia di indici discreti e il decoding VQ-GAN di una griglia di indici nell'immagine RGB finale.

`encode_methods.py`: implementa la classe `SteganographyEncoder`, che realizza la fase di encoding steganografico descritta nell'[sezione 4.5](#). Gestisce l'inizializzazione dei tensori, l'iterazione sulle posizioni della griglia, la selezione dei token e la verifica di codifica.

`decode_methods.py`: implementa la classe `SteganographyDecoder`, che realizza la fase di decoding steganografico descritta nell'[sezione 4.6](#). Ricostruisce le distribuzioni di probabilità percorrendo la griglia e recupera i bit codificati.

## 5.2 LE CLASSI STEGANOGRAPHY ENCODER E STEGANOGRAPHY DECODER

### 5.2.1 *SteganographyEncoder*

La classe `SteganographyEncoder` coordina la generazione guidata dal messaggio. Il suo metodo principale `encode_message_to_image()` implementa l'[Algoritmo 3](#) nei seguenti passi:

1. Estrae l'immagine di contesto dal dataset condiviso e la codifica tramite l'encoder VQ-GAN (`encode_to_z()`) per ottenere la matrice di riferimento **R**.
2. Inizializza il tensore in costruzione **B** copiando le prime R righe da **R** e azzerando le restanti posizioni.
3. Converte il messaggio in sequenza binaria **b**.
4. Itera sulle posizioni (r, c) della griglia in ordine raster: per ciascuna, costruisce il contesto **c** come descritto nella [sezione 4.7](#), invoca la selezione steganografica del token ([Algoritmo 1](#)) ed esegue la verifica di codifica ([Algoritmo 2](#)).
5. Quando **b** è esaurito, completa le posizioni rimanenti con campionamento casuale per garantire la coerenza visiva dell'immagine.
6. Decodifica **B** tramite il decoder VQ-GAN (`decode_to_img()`) per ottenere l'immagine finale e la salva su disco.



### 5.2.2 *SteganographyDecoder*

La classe *SteganographyDecoder* implementa l'[Algoritmo 5](#). Il suo metodo `decode_message()` percorre la griglia nello stesso ordine raster dell'encoding steganografico: per ciascuna posizione  $(r, c)$ , ricostruisce il contesto  $c$ , estrae il token  $z^*$  dall'immagine ricevuta tramite l'encoder VQ-GAN (`encode_to_z()`), e invoca la decodifica steganografica del token ([Algoritmo 4](#)) per recuperare i bit. I bit così ottenuti vengono concatenati e riconvertiti nel messaggio originale.

## 5.3 COSTRUZIONE DEL CONTESTO

Come illustrato nella [sezione 4.7](#), il Transformer opera su una finestra locale  $p \times p$ . L'implementazione di questa costruzione si articola in due funzioni ausiliarie. `local_indexes()` calcola le coordinate della finestra  $p \times p$  centrata sulla posizione corrente, gestendo i casi di bordo mediante traslazione della finestra per mantenerla interamente entro i limiti della griglia. `build_context_from_patches()` utilizza tali coordinate per estrarre le porzioni corrispondenti dalla matrice di riferimento  $\mathbf{R}$  e dal tensore in costruzione  $\mathbf{B}$ , concatenandole in un'unica sequenza di  $2 \times p \times p$  token da fornire al Transformer.

## 5.4 GENERAZIONE E SALVATAGGIO DELLE IMMAGINI

### 5.4.1 *Verifica di codifica*

Un aspetto implementativo critico è il meccanismo di verifica descritto nell'[Algoritmo 2](#). L'implementazione esegue un ciclo di tentativi in cui per ogni token selezionato  $i^*$ :

1. Inserisce  $i^*$  nel tensore  $\mathbf{B}$  alla posizione corrente;
2. Decodifica  $\mathbf{B}$  in un'immagine parziale tramite il decoder VQ-GAN (`decode_to_img()`);
3. Ricodifica l'immagine parziale tramite l'encoder VQ-GAN (`encode_to_z()`);
4. Confronta l'indice ricostruito  $\hat{i}$  con  $i^*$ .

Se  $\hat{i} = i^*$  la codifica è considerata valida; altrimenti si ripete con un nuovo tentativo. Dopo 100 tentativi falliti, la selezione viene forzata per evitare un ciclo infinito, accettando il token corrente anche in caso di discordanza. Questo costo computazionale, come discusso nella [sezione 4.8](#), rende l'encoding steganografico significativamente più oneroso del decoding steganografico.

#### 5.4.2 Salvataggio in formato PNG

Le immagini generate vengono salvate in formato PNG (Portable Network Graphics). Tale scelta non è accidentale: il formato PNG è *lossless* e supporta una profondità di colore di 8 bit per canale (24 bit totali per pixel in RGB), preservando integralmente ogni valore dei tre canali di colore senza perdita di informazione. Ciò è essenziale per la corretta decodifica steganografica del messaggio, poiché qualsiasi compressione con perdita — come quella del formato JPEG, che opera una quantizzazione delle componenti di frequenza nello spazio YCbCr — altererebbe i valori dei pixel e, di conseguenza, gli indici discreti ottenuti dalla ricodifica tramite l'encoder VQ-GAN (`encode_to_z()`), compromettendo il recupero del messaggio. Il formato PNG garantisce invece che l'immagine generata possa essere salvata, trasmessa e ricaricata con degradazione minimale. Non è stata effettuata un'analisi empirica della resitenza del sistema a formati con perdita: un tale studio potrebbe essere oggetto di ricerche future.

### 5.5 LIMITAZIONI PRATICHE

Oltre alle limitazioni teoriche discusse nella [sezione 4.8](#), l'implementazione presenta alcune criticità.

#### 5.5.1 Costi computazionali

Il ciclo di verifica della codifica rende il processo di encoding steganografico computazionalmente oneroso: ogni token può richiedere fino a 100 cicli di decoding VQ-GAN e successivo encoding VQ-GAN dell'immagine parziale. Per un'immagine  $256 \times 256$  con circa 240 token da codificare, il numero di operazioni coinvolte può diventare significativo, con un rapporto asimmetrico tra i tempi di encoding steganografico e decoding steganografico.

### 5.5.2 *Gestione dei casi di bordo*

La traslazione della finestra di contesto vicino ai bordi della griglia, pur garantendo che la finestra sia sempre di dimensione  $p \times p$ , sacrifica la simmetria centrata sulla posizione corrente per le celle prossime al bordo.

### 5.5.3 *Riproducibilità dei risultati*

L'uso di un seme pseudo-casuale fisso garantisce la riproducibilità del campionamento casuale nelle posizioni non steganografiche. Tuttavia, operazioni di arrotondamento e quantizzazione nell'encoder VQ-GAN possono introdurre variazioni minime tra esecuzioni su hardware diverse (diverse implementazioni GPU di operazioni floating-point).



---

## RISULTATI SPERIMENTALI

---

Le peculiarità del modello proposto, se paragonato con i più recenti lavori affini, risiedono primariamente nella inesigenza di questo ad addestrare un modello steganografico ad hoc, atto che porta con sé il rischio di alterare la distribuzione di probabilità delle architetture basali esistenti rendendosi quindi vulnerabile a tecniche di steganalisi, e secondariamente nell'utilizzo di un'architettura Image-to-Image quale è VQ-GAN per l'implementazione del generatore steganografico. Questa seconda peculiarità è risultata uno svantaggio a livello implementativo in quanto, sebbene il modello nel suo insieme possa essere inteso quale SwE (Steganography without Embedding), utilizzi in realtà un dataset.

This is where you show that the novel 'thing' you described in Chapter [Capitolo 4](#) and implemented in Chapter [Capitolo 5](#) is, indeed, much better than the existing versions of the same.

You will probably use figures (try to use a high-resolution version), graphs, tables, and so on. An example is shown in [Figure 2](#).

Note that, likewise tables and listings, you shall not worry about where the figures are placed. Moreover, you should not add the file extension (LaTeX will pick the 'best' one for you) or the figure path.

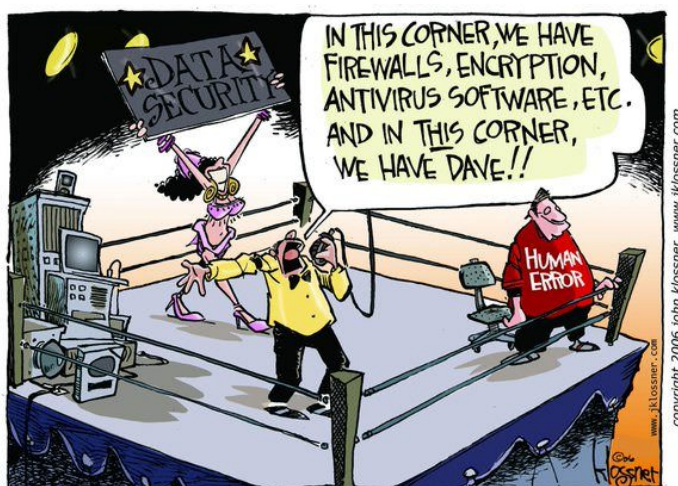


Figura 2: Network Security - the sad truth

---

## CONCLUSIONI E SVILUPPI FUTURI

---

They say that the conclusions are the shortened version of the introduction, and while the Introduction uses future verbs (we will), the conclusions use the past verbs (we did). It is basically true.

In the conclusions, you might also mention the shortcomings of the present work and outline what are the likely, necessary, extension of it. E.g., we did analyse the performance of this network assuming that all the users are pedestrians, but it would be interesting to include in the study also the ones using bicycles or skateboards.

Finally, you are strongly encouraged to carefully spell check your text, also using automatic tools (like, e.g., Grammarly<sup>1</sup> for English language).

---

<sup>1</sup> <https://www.grammarly.com/>





---

## BIBLIOGRAFIA

---

- [1] European Pirate Party. *Chat control 2.0: the sequel nobody asked for*. URL: <https://europeanpirates.eu/chatcontrol-the-sequel-nobody-asked-for/>.
- [2] Gabriel Kaptchuk et al. *Meteor. Cryptographically secure steganography for realistic distributions*. URL: <https://meteorfrom.space/> (visitato il giorno 10/07/2025).
- [3] Matteo Suanno. «Cosa significa che Starlink non è più accessibile all'esercito russo in Ucraina». In: (6 feb. 2026). URL: <https://www.wired.it/article/cosa-sappiamo-blocco-starlink-militari-ucraina/>.
- [4] Dave Bergmann e Cole Stryker. *Diffusion models*. 17 Apr. 2025. URL: <https://www.ibm.com/think/topics/diffusion-models> (visitato il giorno 10/07/2025).
- [5] Yunyun Dong et al. «Double-Flow-Based Steganography Without Embedding for Image-to-Image Hiding». In: *Electronics* 14 (ott. 2025), p. 4270. DOI: [10.3390/electronics14214270](https://doi.org/10.3390/electronics14214270) (cit. a p. 26).
- [6] Ari Ibrahim Hamid e Wafaa Mustafa Abdualлах. «StyleGAN2-STEGO: Secure coverless image steganography via latent space encoding». In: *East Journal of Computer Science* 1.4 (30 ago. 2025). DOI: [10.63496/ejcs.vol1.iss4.188](https://doi.org/10.63496/ejcs.vol1.iss4.188). URL: <https://doi.org/10.63496/ejcs.vol1.iss4.188> (cit. a p. 26).
- [7] Xiao Li et al. «Coverless Image Steganography Based on Semantic-Controlled Text-to-Image Generation». In: *IEEE Transactions on Circuits and Systems for Video Technology* 35.8 (ago. 2025), pp. 8391–8405. ISSN: 1558-2205. DOI: [10.1109/TCSVT.2025.3545067](https://doi.org/10.1109/TCSVT.2025.3545067) (cit. a p. 26).
- [8] Corey Neskey. *Are Your Passwords in the Green?* Hive Systems. 2025. URL: <https://www.hivesystems.com/blog/are-your-passwords-in-the-green> (cit. a p. 30).
- [9] Wisdom Sablah. *WhatsApp Banned Countries List and How to Bypass the Ban 2026*. 13 Nov. 2025. URL: <https://www.cloudwards.net/countries-where-whatsapp-is-banned/>.

- [10] Lina Tan et al. «An Adaptive JPEG Steganography Algorithm Based on the UT-GAN Model». In: *Electronics* 14.20 (2025). ISSN: 2079-9292. DOI: [10.3390/electronics14204046](https://doi.org/10.3390/electronics14204046). URL: <https://www.mdpi.com/2079-9292/14/20/4046> (cit. a p. 25).
- [11] Xiangkun Wang et al. *GIFDL: Generated Image Fluctuation Distortion Learning for Enhancing Steganographic Security*. 2025. arXiv: [2504.15139 \[cs.CR\]](https://arxiv.org/abs/2504.15139). URL: <https://arxiv.org/abs/2504.15139> (cit. a p. 25).
- [12] Yuhua Xu et al. «Approximate Gaussian Mapping for Generative Image Steganography». In: *arXiv preprint arXiv:2510.07219* (2025) (cit. a p. 26).
- [13] Pengbiao Zhao et al. «RLL-SWE: A Robust Linked List Steganography Without Embedding for intelligence networks in smart environments». In: *Journal of Network and Computer Applications* 234 (2025), p. 104053. ISSN: 1084-8045. DOI: <https://doi.org/10.1016/j.jnca.2024.104053>. URL: <https://www.sciencedirect.com/science/article/pii/S1084804524002303> (cit. a p. 26).
- [14] Richard Apau et al. «Image steganography techniques for resisting statistical steganalysis attacks: A systematic literature review». In: *PLOS ONE* 19.9 (set. 2024), pp. 1–47. DOI: [10.1371/journal.pone.0308807](https://doi.org/10.1371/journal.pone.0308807). URL: <https://doi.org/10.1371/journal.pone.0308807> (cit. alle pp. 25, 31).
- [15] Dongxia Huang et al. «Steganography Embedding Cost Learning With Generative Multi-Adversarial Network». In: *IEEE Transactions on Information Forensics and Security* 19 (2024), pp. 15–29. ISSN: 1556-6021. DOI: [10.1109/TIFS.2023.3318939](https://doi.org/10.1109/TIFS.2023.3318939) (cit. a p. 25).
- [16] Li Li et al. «Image Steganography and Style Transformation Based on Generative Adversarial Network». In: *Mathematics* 12.4 (2024). ISSN: 2227-7390. DOI: [10.3390/math12040615](https://doi.org/10.3390/math12040615). URL: <https://www.mdpi.com/2227-7390/12/4/615> (cit. a p. 26).
- [17] Waheed Rehman. *A Novel Approach to Image Steganography Using Generative Adversarial Networks*. 2024. arXiv: [2412.00094 \[cs.CR\]](https://arxiv.org/abs/2412.00094). URL: <https://arxiv.org/abs/2412.00094> (cit. a p. 25).
- [18] Simone Scardapane. *Alice's Adventures in a Differentiable Wonderland – Volume I, A Tour of the Land*. 26 Apr. 2024. URL: <https://arxiv.org/abs/2404.17625>.

- [19] Wenkang Su, Jiangqun Ni e Yiyan Sun. «StegaStyleGAN: towards generic and practical generative image steganography». In: *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence*. AAAI'24/IAAI'24/EAAI'24. AAAI Press, 2024. ISBN: 978-1-57735-887-9. DOI: [10.1609/aaai.v38i1.27776](https://doi.org/10.1609/aaai.v38i1.27776). URL: <https://doi.org/10.1609/aaai.v38i1.27776> (cit. a p. 26).
- [20] Zhili Zhou et al. «Latent Vector Optimization-Based Generative Image Steganography for Consumer Electronic Applications». In: *IEEE Transactions on Consumer Electronics* 70.1 (feb. 2024), pp. 4357–4366. ISSN: 1558-4127. DOI: [10.1109/TCE.2024.3354824](https://doi.org/10.1109/TCE.2024.3354824) (cit. a p. 26).
- [21] Ambika, Virupakshappa e Deepak S. Uplaonkar. «Deep Learning-Based Coverless Image Steganography on Medical Images Shared via Cloud». In: *Engineering Proceedings* 59.1 (2023). ISSN: 2673-4591. DOI: [10.3390/engproc2023059176](https://doi.org/10.3390/engproc2023059176). URL: <https://www.mdpi.com/2673-4591/59/1/176> (cit. a p. 26).
- [22] Giulia Bertazzini. «An overview of Diffusion Models in Multimedia Forensics: Generating and Detecting Synthetic Images». 27 Ott. 2023.
- [23] Kemal Erdem. *Step by step visual introduction to diffusion models*. Nov. 2023. URL: <https://erdem.pl/2023/11/step-by-step-visual-introduction-to-diffusion-models> (visitato il giorno 10/07/2025).
- [24] Ziping Ma et al. «Robust Steganography without Embedding Based on Secure Container Synthesis and Iterative Message Recovery». In: *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23*. A cura di Edith Elkind. Main Track. International Joint Conferences on Artificial Intelligence Organization, ago. 2023, pp. 4838–4846. DOI: [10.24963/ijcai.2023/538](https://doi.org/10.24963/ijcai.2023/538). URL: <https://doi.org/10.24963/ijcai.2023/538> (cit. a p. 26).
- [25] Daniel Bourke. *Learn PyTorch for deep learning in a day. Literally*. 24 Lug. 2022. URL: [https://www.youtube.com/watch?v=Z\\_ikDlimN6A](https://www.youtube.com/watch?v=Z_ikDlimN6A).
- [26] Prafulla Dhariwal e Alex Nichol. «Diffusion Models Beat GANs on Image Synthesis». In: *CoRR abs/2105.05233* (2021). arXiv: [2105.05233](https://arxiv.org/abs/2105.05233). URL: <https://arxiv.org/abs/2105.05233>.

- [27] Shuyang Gu et al. «Vector Quantized Diffusion Model for Text-to-Image Synthesis». In: *CoRR* abs/2111.14822 (2021). arXiv: [2111.14822](https://arxiv.org/abs/2111.14822). URL: <https://arxiv.org/abs/2111.14822>.
- [28] Gabriel Kaptchuk et al. «Meteor: Cryptographically Secure Steganography for Realistic Distributions». In: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. CCS '21. Virtual Event, Republic of Korea: Association for Computing Machinery, 2021*, pp. 1529–1548. ISBN: 9781450384544. DOI: [10.1145/3460120.3484550](https://doi.org/10.1145/3460120.3484550). URL: <https://doi.org/10.1145/3460120.3484550> (cit. alle pp. 11, 27, 44, 53).
- [29] Nandhini Subramanian et al. «Image Steganography: A Review of the Recent Advances». In: *IEEE Access* 9 (2021), pp. 23409–23423. ISSN: 2169-3536. DOI: [10.1109/ACCESS.2021.3053998](https://doi.org/10.1109/ACCESS.2021.3053998).
- [30] Lilian Weng. *What are Diffusion Models?* 11 Lug. 2021. URL: <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>.
- [31] Cong Yu et al. «An improved steganography without embedding based on attention GAN». In: *Peer-to-Peer Networking and Applications* 14 (mag. 2021), pp. 1–12. DOI: [10.1007/s12083-020-01033-x](https://doi.org/10.1007/s12083-020-01033-x) (cit. a p. 26).
- [32] Patrick Esser, Robin Rombach e Björn Ommer. «Taming Transformers for High-Resolution Image Synthesis». In: *CoRR* abs/2012.09841 (2020). arXiv: [2012.09841](https://arxiv.org/abs/2012.09841). URL: <https://arxiv.org/abs/2012.09841> (cit. alle pp. 20–22, 24, 38, 43, 53).
- [33] Jia Liu et al. «Recent Advances of Image Steganography With Generative Adversarial Networks». In: *IEEE Access* 8 (2020), pp. 60575–60597. ISSN: 2169-3536. DOI: [10.1109/ACCESS.2020.2983175](https://doi.org/10.1109/ACCESS.2020.2983175).
- [34] Gianmarco Turchetta. «Applicazioni blockchain in sistemi di Telemedicina: condivisione e accesso sicuro ai dati sanitari». Tesi di Laurea Magistrale in Ingegneria Biomedica. Politecnico di Torino, 2020. URL: <https://webthesis.biblio.polito.it/17016/> (cit. a p. 30).
- [35] Marc Chaumont. «Deep Learning in steganography and steganalysis from 2015 to 2018». In: *CoRR* abs/1904.01444 (2019). arXiv: [1904.01444](https://arxiv.org/abs/1904.01444). URL: <http://arxiv.org/abs/1904.01444> (cit. a p. 25).
- [36] Zhuo Zhang et al. «Generative Reversible Data Hiding by Image to Image Translation via GANs». In: (2019). arXiv: [1905.02872](https://arxiv.org/abs/1905.02872) [eess.IV]. URL: <https://arxiv.org/abs/1905.02872> (cit. a p. 26).

- [37] Zhuo Zhang et al. «Generative Steganography by Sampling». In: *IEEE Access* 7 (2019), pp. 118586–118597. doi: [10.1109/ACCESS.2019.2920313](https://doi.org/10.1109/ACCESS.2019.2920313) (cit. a p. 26).
- [38] Donghui Hu et al. «A Novel Image Steganography Method via Deep Convolutional Generative Adversarial Networks». In: *IEEE Access* 6 (2018), pp. 38303–38314. doi: [10.1109/ACCESS.2018.2852771](https://doi.org/10.1109/ACCESS.2018.2852771) (cit. a p. 25).
- [39] Adrian Shahbaz. *The rise of digital authoritarianism*. 2018. URL: <https://freedomhouse.org/report/freedom-net/2018/rise-digital-authoritarianism>.
- [40] Pin Wu, Yang Yang e Xiaoqiang Li. «StegNet: Mega Image Steganography Capacity with Deep Convolutional Network». In: *Future Internet* 10.6 (giu. 2018), p. 54. ISSN: 1999-5903. doi: [10.3390/fi10060054](https://doi.org/10.3390/fi10060054). URL: <http://dx.doi.org/10.3390/fi10060054> (cit. a p. 25).
- [41] Richard Zhang et al. «The Unreasonable Effectiveness of Deep Features as a Perceptual Metric». In: *CoRR abs/1801.03924* (2018). arXiv: [1801.03924](https://arxiv.org/abs/1801.03924). URL: <http://arxiv.org/abs/1801.03924>.
- [42] Shumeet Baluja. «Hiding Images in Plain Sight: Deep Steganography». In: *Advances in Neural Information Processing Systems*. A cura di I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/838e8afb1ca34354ac209f53d90c3a43-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/838e8afb1ca34354ac209f53d90c3a43-Paper.pdf) (cit. a p. 25).
- [43] Phillip Isola et al. «Image-to-Image Translation with Conditional Adversarial Networks». In: *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*. IEEE Computer Society, 2017, pp. 5967–5976. doi: [10.1109/CVPR.2017.632](https://doi.org/10.1109/CVPR.2017.632). URL: <https://doi.org/10.1109/CVPR.2017.632> (cit. a p. 22).
- [44] Xudong Mao et al. «Least Squares Generative Adversarial Networks». In: *IEEE International Conference on Computer Vision, ICCV 2017, Venice, Italy, October 22-29, 2017*. IEEE Computer Society, 2017, pp. 2813–2821. doi: [10.1109/ICCV.2017.304](https://doi.org/10.1109/ICCV.2017.304). URL: <https://doi.org/10.1109/ICCV.2017.304>.
- [45] Aäron van den Oord, Oriol Vinyals e Koray Kavukcuoglu. «Neural Discrete Representation Learning». In: *CoRR abs/1711.00937* (2017). arXiv: [1711.00937](https://arxiv.org/abs/1711.00937). URL: <http://arxiv.org/abs/1711.00937> (cit. alle pp. 20, 38).

- [46] Haichao Shi et al. «SSGAN: Secure Steganography Based on Generative Adversarial Networks». In: *CoRR* abs/1707.01613 (2017). arXiv: [1707.01613](https://arxiv.org/abs/1707.01613). URL: <http://arxiv.org/abs/1707.01613> (cit. a p. 26).
- [47] Ashish Vaswani et al. «Attention is All you Need». In: *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*. 2017, pp. 5998–6008. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html> (cit. a p. 21).
- [48] Kate Conger. *WhatsApp blocked in Brazil again*. 19 Lug. 2016. URL: <https://techcrunch.com/2016/07/19/whatsapp-blocked-in-brazil-again/>.
- [49] Justin Johnson, Alexandre Alahi e Li Fei-Fei. «Perceptual Losses for Real-Time Style Transfer and Super-Resolution». In: *Computer Vision - ECCV 2016 - 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part II*. Vol. 9906. Lecture Notes in Computer Science. Springer, 2016, pp. 694–711. doi: [10.1007/978-3-319-46475-6\\_43](https://doi.org/10.1007/978-3-319-46475-6_43). URL: [https://doi.org/10.1007/978-3-319-46475-6\\_43](https://doi.org/10.1007/978-3-319-46475-6_43).
- [50] Roya Ensafi et al. «Examining How the Great Firewall Discovers Hidden Circumvention Servers». In: *Proceedings of the 2015 Internet Measurement Conference*. IMC '15. Tokyo, Japan: Association for Computing Machinery, 2015, pp. 445–458. ISBN: 9781450338486. DOI: [10.1145/2815675.2815690](https://doi.org/10.1145/2815675.2815690). URL: <https://doi.org/10.1145/2815675.2815690> (cit. a p. 13).
- [51] Karen Simonyan e Andrew Zisserman. «Very Deep Convolutional Networks for Large-Scale Image Recognition». In: *3rd International Conference on Learning Representations (ICLR)*. San Diego, CA, USA, 2015. URL: <https://arxiv.org/abs/1409.1556> (cit. alle pp. 23, 38).
- [52] Ian J. Goodfellow et al. «Generative Adversarial Nets». In: *CoRR* abs/1406.2661 (2014). arXiv: [1406.2661](https://arxiv.org/abs/1406.2661). URL: <http://arxiv.org/abs/1406.2661> (cit. a p. 20).